

Module 19: Advanced Control Systems

19.1 Introduction to Advanced Control

Control System Fundamentals

What is Control?

Control System: A system that manages, commands, directs, or regulates the behavior of other devices or systems to achieve desired outcomes.

CNC Context: Control systems position machine axes accurately and smoothly while following programmed toolpaths.

Open-Loop vs Closed-Loop Control

Open-Loop Control (Stepper Motors):

Command -> Controller -> Motor -> Position
(no feedback)

Characteristics: - No measurement of actual position - Assumes motor follows commands perfectly - Simple, low-cost - Risk: Missed steps undetected

Example: Stepper motor commanded to move 1000 steps - Expected: $1000 \text{ steps} \times 0.0002''/\text{step} = 0.200''$ movement - Actual: If 10 steps missed -> $0.198''$ movement - **Problem:** Controller doesn't know steps were missed

Closed-Loop Control (Servo Motors):

Command -> Controller -> Motor -> Position
up down
<----- Encoder <-----+
(feedback)

Characteristics: - Continuous measurement of actual position - Controller corrects deviations from desired position - Higher cost, more complex - Detects and corrects errors

Example: Servo commanded to move $0.200''$ - Encoder measures actual position continuously - If position lags: Controller increases motor torque - Achieves $0.200'' \pm 0.0001''$ (much more accurate)

Why Closed-Loop?

Advantages: 1. **Higher accuracy:** Feedback compensates for disturbances 2. **Faster motion:** Higher accelerations possible (no resonance issues) 3. **Error detection:** Following errors trigger alarms 4. **Load compensation:** Automatically adjusts for cutting forces 5. **Tunable performance:** Adjust response characteristics

Disadvantages: 1. **Higher cost:** \$500-3000 per axis vs \$50-300 for steppers 2. **Complexity:** Requires tuning and setup 3. **Potential instability:** Poor tuning causes oscillation 4. **Maintenance:** Encoders can fail or drift

When to Use Servos: - High-speed machining (>500 IPM rapids) - Precision requirements ($<0.001''$ positioning) - High acceleration requirements (>150 in/s²) - Variable cutting loads (heavy machining) - Production environment (reliability critical)

When Steppers Sufficient: - Hobby/DIY projects (cost-sensitive) - Light cutting loads (3D printing, laser cutting, light routing) - Moderate speeds (<200 IPM) - Low acceleration requirements (<100 in/s²)

Servo System Components

Motor

Brushed DC Servo: - Commutation via brushes (wearing parts) - Simple to control (voltage $\rightarrow$ speed) - Lower cost (\$100-500) - Maintenance: Brush replacement every 2000-5000 hours

Brushless DC Servo (BLDC): - Electronic commutation (no brushes) - Higher efficiency (85-95% vs 75-85%) - Higher reliability (no brush wear) - Requires more complex drive - Cost: \$300-1500

AC Servo (PMSM - Permanent Magnet Synchronous Motor): - Similar to BLDC, different control algorithm - Highest performance (torque density, efficiency) - Used in industrial applications - Cost: \$500-3000+

Key Specifications: - **Continuous torque:** Torque motor can sustain indefinitely (thermal limit) - **Peak torque:** Maximum torque for short duration (2-3x continuous) - **Speed range:** Typical 0-3000 RPM (some up to 6000 RPM) - **Inertia:** Rotor inertia affects acceleration (lower better for CNC)

Example Motor: 400W BLDC servo - Continuous torque: 1.27 Nm (*180 oz-in*) - *Peak torque: 3.82 Nm* (540 oz-in) - Rated speed: 3000 RPM - Rotor inertia: 0.18 kg*cm²

Encoder (Feedback Device)

Incremental Encoder: - Outputs pulses as shaft rotates - A and B channels (quadrature) for direction sensing - Z index pulse once per revolution - Resolution: 1000-10,000 PPR (pulses per revolution) typical - Relative position only (loses position on power-off)

Quadrature Encoding: - A and B channels 90 deg out of phase - Rising/falling edges on both $\rightarrow$ 4x resolution - 1000 PPR encoder $\rightarrow$ 4000 counts per revolution

Example: - 2000-line encoder (4x = 8000 counts/rev) - 5mm pitch ballscrew - Resolution: 5mm / 8000 = 0.000625 mm = 0.625 μ m = 0.000025"

Absolute Encoder: - Outputs actual position (not just increments) - Retains position on power-off - More expensive (\$200-800 vs \$50-200 for incremental) - Used in industrial applications, robotics

Linear Encoder (Glass Scale): - Directly measures linear position (not rotary) - Eliminates errors from ballscrew pitch variation, backlash - Higher cost (\$300-1500 per axis) - Precision applications ($<0.0001''$ positioning)

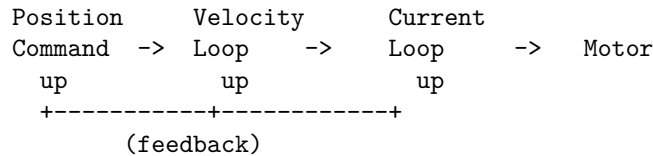
Resolver: - Analog position sensor (AC excited) - Extremely rugged (no optical parts) - Lower resolution than encoder - Used in harsh environments

Servo Drive (Amplifier)

Function: Converts control signals (position commands) into motor power.

Typical Drive Specifications: - Input: DC bus voltage (24-340 VDC typical) - Output: 3-phase PWM to motor (for BLDC/AC servo) - Current rating: 5-30A continuous, 15-90A peak - Control modes: Position, velocity, torque - Communication: Analog (± 10 V), step/direction, EtherCAT, CANopen

Control Loop: Modern servo drives implement cascaded control:



Example Drive: 750W AC Servo Drive - Input voltage: 220 VAC single-phase (rectified to 310 VDC bus)
 - Output: 3-phase, 0-220 VAC (PWM) - Continuous current: 3.4A - Peak current: 10.2A (3x overload) -
 Position loop frequency: 1-2 kHz - Current loop frequency: 8-16 kHz

Controller

CNC Controller Functions: 1. **Interpret G-code:** Parse commands, generate trajectories 2. **Trajectory planning:** Calculate smooth motion profiles 3. **Position control:** PID loops for each axis 4. **Interpolation:** Coordinate multi-axis motion 5. **I/O management:** Spindle, coolant, tool changer control

Real-Time Requirements: - Position loop update: 1-2 kHz typical (every 0.5-1 ms) - Jitter: <100 mus (deterministic timing critical) - Latency: <1 ms (command to motion delay)

Controller Options: - **LinuxCNC:** PC-based, open-source, real-time Linux kernel - **Mach4:** PC-based, commercial, motion plugin architecture - **Dedicated controller:** FANUC, Siemens, Haas (industrial, closed-source) - **Arduino/Teensy:** DIY, limited performance (<1 kHz loop rates typically)

Performance Metrics

Following Error

Definition: Difference between commanded position and actual position during motion.

$$\text{Following Error} = \text{Position}_{\text{commanded}} - \text{Position}_{\text{actual}}$$

Sources: 1. **Proportional gain too low:** Motor doesn't respond fast enough 2. **Velocity feedforward insufficient:** Constant lag during motion 3. **Friction:** Stiction causes position lag 4. **Mechanical compliance:** Frame/coupling flex under load

Example: System moving at 100 IPM (1.67 in/s): - With velocity feedforward: Following error = 0.0005" (good) - Without velocity feedforward: Following error = 0.005" (poor)

Acceptable Limits: - High-speed machining: <0.001" during rapids - Precision machining: <0.0005" during cutting - Heavy machining: <0.002" acceptable (cutting forces dominate)

Following Error Alarm: When following error exceeds threshold (0.010-0.050" typical), controller issues alarm and stops motion.

Typical scenario: - Axis encounters obstruction (crash, over-torque) - Motor can't overcome resistance - Position lags further and further behind - Following error exceeds limit -> ALARM

Settling Time

Definition: Time for position to reach and stay within specified tolerance band after commanded move.

Standard Specification: Time to settle within +/-5% of final value (sometimes +/-2% or +/-1%)

Example: - Command: Move 1.000" - Target band: 1.000" +/- 0.005" (+/-0.5%) - Settling time: Time from start until position remains in 0.995-1.005" range

Typical Values: - Underdamped system (oscillatory): 100-300 ms - Critically damped: 50-100 ms - Overdamped (sluggish): 200-500 ms

Effect on Cycle Time: Shorter settling time = faster point-to-point moves.

For 100 point-to-point moves per part: - 200 ms settling: 20 seconds wasted - 50 ms settling: 5 seconds wasted - **Savings:** 15 seconds per part

At 10 parts/hour: 2.5 minutes saved per hour = 4% cycle time reduction

Rise Time

Definition: Time to reach target position (measured from 10% to 90% of final value).

Fast rise time -> high acceleration -> high throughput

Trade-off: Very fast rise time risks overshoot and oscillation.

Overshoot

Definition: Amount by which response exceeds final value before settling.

$$\text{Percent Overshoot} = \frac{\text{Peak Value} - \text{Final Value}}{\text{Final Value}} \times 100\%$$

Example: - Commanded move: 1.000" - Peak position: 1.015" - Overshoot: 0.015" / 1.000" = 1.5%

Acceptable Overshoot: - None: Critical positioning (probing, part handling) - <5%: Precision machining - <10%: General machining - <25%: Non-critical positioning

Zero Overshoot: Critically damped or overdamped system (slower response)

Bandwidth

Definition: Frequency at which closed-loop gain drops to -3 dB (70.7% of DC gain).

Practical Meaning: Maximum frequency of position commands system can follow accurately.

Typical CNC Servo Bandwidth: 20-100 Hz

Example: - Bandwidth: 50 Hz - System can follow sinusoidal position commands up to ~50 Hz - At 100 Hz (2x bandwidth): Response significantly attenuated

Higher Bandwidth -> Faster response -> Better contour accuracy during high-speed curves

Bandwidth Limitations: 1. Mechanical resonances (typically 100-500 Hz) 2. Control loop update rate (1 kHz loop -> ~100 Hz max bandwidth) 3. Motor/drive response time

Open-Loop vs Closed-Loop Trade-Offs

Accuracy Comparison

Stepper Open-Loop (typical 1/8 microstepping): - Resolution: 0.0001-0.0005" (depends on screw pitch) - Repeatability: +/-0.001-0.005" (depends on missed steps) - Absolute accuracy: +/-0.005-0.020" (cumulative errors)

Servo Closed-Loop (2000-line encoder): - Resolution: 0.00002-0.00005" - Repeatability: +/-0.0001-0.0005" - Absolute accuracy: +/-0.0005-0.002" (limited by encoder mounting/coupling)

Linear Encoder Servo: - Resolution: 0.00002-0.00005" (0.5-1 μm) - Repeatability: +/-0.00005-0.0002" - Absolute accuracy: +/-0.0001-0.0005" (limited by mechanical loop)

Speed Comparison

Stepper: - Typical max speed: 100-300 IPM (limited by torque drop-off) - Acceleration: 50-100 in/s² (limited by resonance)

Servo: - Typical max speed: 300-1000+ IPM (motor-limited) - Acceleration: 200-500 in/s² (motor torque-limited)

Time Savings Example: 1" rapid move:

Stepper (150 IPM = 2.5 in/s, 75 in/s² accel): - Accel distance: $v^2/(2a) = 2.5^2/(2 \times 75) = 0.042"$ - Accel time: $v/a = 2.5/75 = 0.033$ s - Constant velocity distance: $1.0 - 2 \times 0.042 = 0.916"$ - Constant velocity time: $0.916/2.5 = 0.366$ s - Total time: $2 \times 0.033 + 0.366 = 0.432$ s

Servo (600 IPM = 10 in/s, 300 in/s² accel): - Accel distance: $10^2/(2 \times 300) = 0.167"$ - Accel time: $10/300 = 0.033$ s - Constant velocity distance: $1.0 - 2 \times 0.167 = 0.666"$ - Constant velocity time: $0.666/10 = 0.067$ s - Total time: $2 \times 0.033 + 0.067 = 0.133$ s

Servo is 3.2x faster for this rapid move!

For 50 rapids per part: 15 seconds saved (stepper) vs 6.7 seconds (servo) = 8.3 s savings per part

Cost Comparison (Single Axis)

Stepper System: - Motor: \$50-150 - Driver: \$30-100 - Power supply: \$30-80 - **Total: \$110-330 per axis**

Basic Servo System: - Motor with encoder: \$200-500 - Servo drive: \$250-800 - Power supply: \$50-150 - **Total: \$500-1450 per axis**

High-Performance Servo: - Motor with high-res encoder: \$400-1000 - Industrial servo drive: \$500-2000 - Power supply: \$100-300 - **Total: \$1000-3300 per axis**

Linear Scale Servo (ultimate accuracy): - Motor with encoder: \$200-500 - Linear scale (glass): \$300-1500 - Servo drive: \$500-1500 - **Total: \$1000-3500 per axis**

Cost-Benefit Analysis

When Steppers Make Sense: - DIY/hobby projects (budget < \$2000 total) - Light cutting (3D printing, laser, light routing) - Cycle time not critical - Moderate precision requirements (+/-0.005")

When Servos Justified: - Commercial/production use - Cycle time reduction pays for itself (months to 1-2 years) - Precision requirements (+/-0.001" or better) - High-speed machining (>400 IPM rapids)

ROI Example: Small production shop, \$75/hour machine rate: - Servo upgrade cost: 3 axes x \$800 = \$2400 - Cycle time reduction: 25% (from 8 min to 6 min per part) - Parts per day (8 hrs): 60 parts (stepper) -> 80 parts (servo) - Additional revenue: 20 parts/day x \$20 margin = \$400/day - **ROI:** 2400 / 400 = 6 days!

(Of course, bottlenecks elsewhere may prevent full realization of time savings)

System Dynamics Overview

Second-Order System Model

Most mechanical systems approximate second-order response:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

where: - ω_n = natural frequency (rad/s) - ζ (zeta) = damping ratio

Damping Ratio Effects:

Underdamped ($\zeta < 1$): - Oscillatory response - Overshoot present - Fast rise time - Example: $\zeta = 0.5 \rightarrow$ 16% overshoot

Critically Damped ($\zeta = 1$): - Fastest response without overshoot - Optimal for many CNC applications - No oscillation

Overdamped ($\zeta > 1$): - Sluggish response - No overshoot - Long settling time - Too conservative for most CNC

Typical CNC Target: $\zeta = 0.7-0.9$ (slightly underdamped, minimal overshoot, fast response)

Natural Frequency

Definition: Frequency at which system oscillates if disturbed with no damping.

CNC Context: Higher natural frequency $\rightarrow$ faster response $\rightarrow$ higher bandwidth

Factors Affecting Natural Frequency: 1. **Mechanical stiffness:** Stiffer machine $\rightarrow$ higher ω_n 2. **Moving mass:** Heavier axis $\rightarrow$ lower ω_n 3. **Control gains:** Higher gains $\rightarrow$ effectively higher ω_n

Example: - Stiff machine: $\omega_n = 60$ rad/s (9.5 Hz) - Flexible machine: $\omega_n = 20$ rad/s (3.2 Hz)

Stiff machine can achieve 3x faster response!

Why Mechanical Design Matters: No amount of control tuning compensates for poor mechanical design. Build it stiff!

Introduction to PID Control

PID: Proportional-Integral-Derivative

Function: Calculate motor command based on position error.

Block Diagram:

```
Error -> [ P ] -> +
      down   [ I ] -> + -> Motor Command
      down   [ D ] -> +
```

Error = Commanded Position - Actual Position

Proportional (P)

Formula: $u_P = K_P \times e$

where: - u_P = proportional output - K_P = proportional gain - e = error

Effect: Output proportional to current error - Large error $\rightarrow$ large correction - Small error $\rightarrow$ small correction

Problem: Steady-state error remains (P-only can't eliminate error completely)

Example: - $K_P = 100$ - Error = 0.010" - Output = $100 \times 0.010 = 1.0$ (some units, e.g., volts or %)

Integral (I)

Formula: $u_I = K_I \times \int e dt$

Effect: Accumulates error over time - Eliminates steady-state error - Corrects for constant disturbances (friction, gravity)

Problem: Can cause overshoot and oscillation if too high

Example: - Error persists at 0.001" for 0.1 seconds - $K_I = 500$ - Integral term accumulates: $500 \times 0.001 \times 0.1 = 0.05$

Derivative (D)

Formula: $u_D = K_D \times \frac{de}{dt}$

Effect: Responds to rate of change of error - Anticipates future error - Provides damping (reduces overshoot)

Problem: Amplifies high-frequency noise

Example: - Error changing at 0.1 in/s (rapidly decreasing) - $K_D = 10$ - Output = $10 \times 0.1 = 1.0$ (opposes rapid change)

Combined PID

$$u = K_P e + K_I \int e dt + K_D \frac{de}{dt}$$

Tuning Goal: Find K_P , K_I , K_D values that achieve: - Fast response (high K_P , K_D) - No steady-state error (K_I eliminates) - Minimal overshoot (balanced K_P , K_D) - Stable (gains not too high)

Preview of Advanced Techniques

Feedforward Control

Limitation of Feedback-Only: PID reacts to error (inherently lagging)

Feedforward Solution: Add term based on commanded velocity/acceleration - Anticipates required motor torque - Reduces following error during motion

Example: - Without feedforward: 0.005" following error during 200 IPM motion - With velocity feedforward: 0.0005" following error (10x improvement)

Notch Filters

Problem: Mechanical resonances excite oscillation

Solution: Notch filter rejects specific frequency - Identify resonance (e.g., 247 Hz) - Apply notch filter at 247 Hz - Eliminates resonance from control loop

Result: Can increase gains (faster response) without exciting resonance

State-Space Control

Modern Control Theory: Model system as set of first-order differential equations

Advantages: - Handle multi-input, multi-output systems - Optimal controller design (LQR - Linear Quadratic Regulator) - State estimation (Kalman filter)

CNC Application: Mainly research/advanced industrial systems (LinuxCNC uses classical PID)

Summary

Advanced control systems enable high-performance CNC machines through:

1. **Closed-loop feedback:** Continuous error correction for accuracy
2. **PID control:** Tunable response characteristics
3. **Advanced techniques:** Feedforward, filtering, trajectory optimization
4. **Performance metrics:** Quantify and optimize system behavior

Next Steps: - Learn control theory fundamentals (Section 19.2) - Master PID tuning (Sections 19.3-19.4) - Implement trajectory planning (Sections 19.6-19.9) - Configure real systems (Sections 19.10-19.11)

Key Takeaway: Good mechanical design + proper servo tuning = high-performance CNC machine

Next: 19.2 Control System Theory

19.10 Implementation in LinuxCNC

LinuxCNC Overview

LinuxCNC (formerly EMC2): Open-source CNC control software running on Linux with real-time kernel (RTAI or Preempt-RT).

Key Features: - Real-time trajectory planning (1-2 kHz) - HAL (Hardware Abstraction Layer) for flexible configuration - Built-in PID control loops - Extensive servo tuning tools (Halscope, Halshow) - Support for steppers and servos - Free and open-source

Architecture:

```
G-Code -> Interpreter -> Trajectory Planner -> Motion Controller
                                     down
                                     HAL (connections)
                                     down
                                PID Loops -> Motor Drivers
                                     up
                                Encoders (feedback)
```

HAL Configuration Basics

What is HAL?

Hardware Abstraction Layer (HAL): Flexible system for connecting components.

Components: - **Pins:** Inputs/outputs (like physical pins on IC) - **Signals:** Connections between pins (wires) - **Parameters:** Adjustable values (gains, limits) - **Functions:** Code executed periodically (PID loop, encoder reading)

Analogy: Breadboard for CNC control (connect components with virtual wires).

HAL File Structure

Main Configuration Files: - **machine.ini:** High-level machine configuration (axes, limits, tuning) - **machine.hal:** HAL component connections - **custom.hal:** User customizations (optional) - **postgui.hal:** HAL commands after GUI loads (optional)

Basic HAL Commands

Load Component:

```
loadrt pid names=pid.x,pid.y,pid.z
```

Add Function to Thread:

```
addf pid.x.do-pid-calcs servo-thread
```

Connect Pins (create signal):

```
net xpos-cmd axis.x.motor-pos-cmd => pid.x.command
net xpos-fb encoder.0.position => pid.x.feedback
```

Set Parameter:

```
setp pid.x.Pgain 100
setp pid.x.Igain 0.5
setp pid.x.Dgain 10
```

Servo Configuration in LinuxCNC

INI File Configuration

[TRAJ] Section (Trajectory Planner):

```
[TRAJ]
COORDINATES = X Y Z
LINEAR_UNITS = inch
ANGULAR_UNITS = degree
DEFAULT_LINEAR_VELOCITY = 1.0
MAX_LINEAR_VELOCITY = 10.0
DEFAULT_LINEAR_ACCELERATION = 20.0
MAX_LINEAR_ACCELERATION = 200.0
```

[AXIS_X] Section (Per-Axis Configuration):

```
[AXIS_X]
TYPE = LINEAR
HOME = 0.0
MAX_VELOCITY = 10.0
MAX_ACCELERATION = 200.0
MIN_LIMIT = -0.01
MAX_LIMIT = 24.01

# Servo tuning
FERROR = 0.050
MIN_FERROR = 0.010

# Scale (encoder counts per machine unit)
SCALE = 8000.0

# Home sequence
HOME_OFFSET = 0.0
HOME_SEARCH_VEL = 0.50
HOME_LATCH_VEL = 0.05
HOME_SEQUENCE = 0
```

Key Parameters: - **MAX_VELOCITY:** Axis maximum velocity (in/s or mm/s) - **MAX_ACCELERATION:** Axis maximum acceleration - **FERROR:** Maximum allowed following error (triggers fault if exceeded) - **MIN_FERROR:** Minimum following error (for stationary axis) - **SCALE:** Encoder counts per machine unit (e.g., 8000 counts/inch)

HAL File for Servo Axis

Example X-Axis Configuration (machine.hal):

```
# Load PID component
loadrt pid names=pid.x

# Load encoder counter
loadrt encoder num_chan=1
setp encoder.0.position-scale 8000
setp encoder.0.counter-mode 0

# Add functions to servo thread
addf encoder.0.capture-position servo-thread
addf pid.x.do-pid-calcs servo-thread
```

```

addf motion-command-handler servo-thread
addf motion-controller servo-thread

# Connect position command from motion controller to PID
net xpos-cmd axis.x.motor-pos-cmd => pid.x.command

# Connect encoder feedback to PID
net xpos-fb encoder.0.position => pid.x.feedback
net xpos-fb => axis.x.motor-pos-fb

# Connect PID output to motor driver (PWM or analog)
net xoutput pid.x.output => pwmgen.0.value

# Connect enable signal
net xenable axis.x.amp-enable-out => pid.x.enable

# Set PID gains
setp pid.x.Pgain 100
setp pid.x.Igain 10
setp pid.x.Dgain 8
setp pid.x.FF1 0.95
setp pid.x.FF2 0.001
setp pid.x.bias 0.0
setp pid.x.deadband 0.0001
setp pid.x.maxoutput 10.0

```

Encoder Configuration

Quadrature Encoder:

```

setp encoder.0.position-scale 8000 # counts per machine unit
setp encoder.0.counter-mode 0      # quadrature mode
setp encoder.0.x4-mode 1           # 4x counting (rising/falling edges)

```

Position Scale Calculation:

$$\text{Scale} = \frac{\text{Encoder CPR} \times 4}{\text{Distance per Rev}}$$

Example: - Encoder: 2000 CPR (counts per revolution) - Quadrature (4x): 8000 counts/rev - Ballscrew pitch: 0.2 in/rev - Scale: $8000 / 0.2 = \mathbf{40,000 \text{ counts/inch}}$

Example 2 (metric): - Encoder: 2000 CPR -> 8000 counts/rev - Ballscrew pitch: 5 mm/rev - Scale: $8000 / 5 = \mathbf{1600 \text{ counts/mm}}$

Motor Driver Configuration

Analog Velocity Drive (+/-10V):

```

setp pwmgen.0.scale 10.0          # +/-10V output range
setp pwmgen.0.offset 0.0          # no offset
net xoutput pid.x.output => pwmgen.0.value

```

Step/Direction Drive (for servos with step input):

```

setp stepgen.0.position-scale 8000 # steps per machine unit
setp stepgen.0.maxvel 10.0          # max velocity (machine units/s)
setp stepgen.0.maxaccel 200.0       # max acceleration

```

```
net xpos-cmd axis.x.motor-pos-cmd => stepgen.0.position-cmd
net xpos-fb stepgen.0.position-fb => axis.x.motor-pos-fb
```

PID Tuning with Halscope

Halscope Setup

Launch Halscope:

```
halscope &
```

Configure Channels: 1. Click “Add Channel” 2. Select signals to monitor: - `axis.x.motor-pos-cmd` (commanded position) - `axis.x.motor-pos-fb` (actual position) - `axis.x.f-error` (following error) - `pid.x.output` (motor command)

Trigger Setup: - Source: `axis.x.motor-pos-cmd` - Level: 0.1 (trigger when position changes) - Edge: Rising

Run: - Click “Run” to capture data - Command axis motion (jog X-axis) - Observe response in Halscope

Tuning Procedure with Halscope

Step 1: Set Initial Gains (Conservative)

```
halcmd setp pid.x.Pgain 50
halcmd setp pid.x.Igain 0
halcmd setp pid.x.Dgain 0
halcmd setp pid.x.FF1 0
```

Step 2: Test Step Response - Jog X-axis 0.5 inches - Observe in Halscope: - Following error magnitude - Settling time - Overshoot

Step 3: Increase P Gain - Increase Pgain by 50% increments - Repeat step response test - Stop when: - Overshoot > 10-15%, or - Oscillation appears

Example:

```
halcmd setp pid.x.Pgain 75
# Test, observe response
halcmd setp pid.x.Pgain 100
# Test, observe response
halcmd setp pid.x.Pgain 150
# Test - too oscillatory, back off
halcmd setp pid.x.Pgain 125 # Final P value
```

Step 4: Add Velocity Feedforward (FF1) - Jog at constant velocity (e.g., 100 IPM) - Observe following error during constant velocity phase - Set FF1 = 0.9, increase to 1.0 - Optimal: Following error < 0.001” during cruise

```
halcmd setp pid.x.FF1 0.90
# Test
halcmd setp pid.x.FF1 0.95
# Test - following error nearly zero
```

Step 5: Add Derivative Gain - Set Dgain = Pgain / 10 (start) - Test step response - Increase until overshoot < 5-10%

```
halcmd setp pid.x.Dgain 10
# Test, still 15% overshoot
halcmd setp pid.x.Dgain 15
# Test, 8% overshoot - good
```

Step 6: Add Integral Gain - Set $I_{gain} = P_{gain} / 20$ (start) - Test step response - Increase until steady-state error eliminated - Watch for increased overshoot

```
halcmd setp pid.x.Igain 5
# Test, steady-state error = 0.0002"
halcmd setp pid.x.Igain 10
# Test, steady-state error < 0.0001" - good
```

Step 7: Add Acceleration Feedforward (FF2) - Command rapid move with high acceleration - Observe following error spike during accel/decel - Set $FF2 = 0.0001$, increase incrementally

```
halcmd setp pid.x.FF2 0.001
# Test, transient error reduced
halcmd setp pid.x.FF2 0.002
# Test, transient error < 0.001" - good
```

Step 8: Save Final Values

Update machine.ini [AXIS_X] section:

```
[AXIS_X]
P = 125
I = 10
D = 15
FF0 = 0
FF1 = 0.95
FF2 = 0.002
BIAS = 0.0
DEADBAND = 0.0001
MAX_OUTPUT = 10.0
```

Advanced HAL Configuration

Notch Filter for Resonance

Load Notch Filter Component:

```
loadrt notch names=notch.x
setp notch.x.freq 247          # Resonance frequency (Hz)
setp notch.x.q 6.67           # Q factor (1/(2*zeta))
addf notch.x servo-thread
```

Insert in PID Output Path:

```
# Before: PID output directly to motor
# net xoutput pid.x.output => pwmgen.0.value
```

```
# After: PID output through notch filter
net x-pid-out pid.x.output => notch.x.in
net xoutput notch.x.out => pwmgen.0.value
```

Effect: Eliminates 247 Hz resonance from control loop, allows higher gains.

Low-Pass Filter on Derivative

Load Low-Pass Filter:

```
loadrt lowpass names=lowpass.x-deriv
setp lowpass.x-deriv.gain 1.0
```

```
setp lowpass.x-deriv.time-constant 0.002 # 2 ms = ~80 Hz cutoff
addf lowpass.x-deriv servo-thread
```

Apply to Encoder Velocity Signal (used by D term):

```
# Encoder velocity to low-pass filter
net x-vel-raw encoder.0.velocity => lowpass.x-deriv.in

# Filtered velocity to PID (for derivative term)
net x-vel-filt lowpass.x-deriv.out => pid.x.command-deriv
```

Effect: Reduces noise amplification by derivative term.

Cross-Coupling for Gantry

Dual-Motor Gantry (Y1, Y2):

Load Gantry Kinematics:

```
loadrt trivkins coordinates=xyyz
```

INI File:

```
[KINS]
KINEMATICS = trivkins coordinates=xyyz
JOINTS = 4
```

```
[JOINT_1] # Y1 (left motor)
TYPE = LINEAR
MAX_VELOCITY = 8.0
# ... (standard config)
```

```
[JOINT_2] # Y2 (right motor)
TYPE = LINEAR
MAX_VELOCITY = 8.0
# ... (standard config)
```

HAL Configuration:

```
# Both joints receive same command from trajectory planner
net y-pos-cmd axis.y.motor-pos-cmd => joint.1.motor-pos-cmd
net y-pos-cmd => joint.2.motor-pos-cmd
```

```
# Individual feedback from each encoder
net y1-pos-fb encoder.1.position => joint.1.motor-pos-fb
net y2-pos-fb encoder.2.position => joint.2.motor-pos-fb
```

```
# Gantry kinematics handles coordination automatically
```

LinuxCNC Automatic Synchronization: Kinematics module keeps motors synchronized.

Trajectory Planner Configuration

INI File Trajectory Settings

[TRAJ] Section:

```
[TRAJ]
COORDINATES = X Y Z
LINEAR_UNITS = inch
```

```
ANGULAR_UNITS = degree
```

```
# Maximum velocities
```

```
DEFAULT_LINEAR_VELOCITY = 2.0    # Default jog/rapid speed
```

```
MAX_LINEAR_VELOCITY = 10.0       # Maximum allowed speed
```

```
# Acceleration
```

```
DEFAULT_LINEAR_ACCELERATION = 50.0
```

```
MAX_LINEAR_ACCELERATION = 200.0
```

```
# Trajectory planning
```

```
NO_FORCE_HOMING = 1              # Allow motion before homing (testing only)
```

```
POSITION_FILE = position.txt     # Save position on shutdown
```

Blending Mode: - G64: Blending mode (set in G-code or GUI) - G64 P[tolerance]: Blend with tolerance

Example G-Code:

```
G64 P0.005 ; Blend corners with max 0.005" deviation
```

```
G1 X10 Y10 F100
```

```
G1 X10 Y20
```

Real-Time Performance Tuning

Check Real-Time Performance:

```
halrun -I
```

```
halcmd loadrt threads period1=1000000 name1=servo-thread
```

```
halcmd start
```

```
halcmd show thread
```

Latency Test (critical for real-time):

```
latency-test
```

Acceptable Latency: - Base thread: < 25,000 ns (25 mus) - Servo thread: < 50,000 ns (50 mus)

High Latency (>100 mus): - Disable power management (CPU frequency scaling) - Disable SMI (System Management Interrupts) - Use dedicated real-time system

Config (/etc/default/grub):

```
GRUB_CMDLINE_LINUX="isolcpus=1 idle=poll"
```

Homing Configuration

Home Sequence

[AXIS_X] Homing Parameters:

```
[AXIS_X]
```

```
HOME = 0.0                                # Home position (machine coordinates)
```

```
HOME_OFFSET = 0.0                        # Offset from home switch to home position
```

```
HOME_SEARCH_VEL = 0.50                   # Fast search velocity
```

```
HOME_LATCH_VEL = 0.05                    # Slow latch velocity (after switch found)
```

```
HOME_USE_INDEX = 1                       # Use encoder index pulse (high precision)
```

```
HOME_IGNORE_LIMITS = 0                   # Don't ignore limit switches during home
```

```
HOME_SEQUENCE = 0                        # Homing order (all axes with 0 home simultaneously)
```

```
HOME_IS_SHARED = 0                       # Shared home switch (multiple axes)
```

Homing Procedure: 1. Move in direction of home switch at HOME_SEARCH_VEL 2. When switch triggers, back off slowly 3. Approach again at HOME_LATCH_VEL 4. If HOME_USE_INDEX = 1, latch on encoder index pulse (high precision) 5. Set position to HOME + HOME_OFFSET

Encoder Index Homing (High Precision)

Why: Encoder index pulse = precise reference point (repeatable to 1 encoder count).

Configuration:

HOME_USE_INDEX = 1

HAL Connection:

```
net x-index-enable encoder.0.index-enable <=> axis.x.index-enable
```

Repeatability: +/-1 encoder count (e.g., 0.000025" for 40,000 count/inch encoder).

Testing and Validation

Step Response Test

Procedure: 1. Home all axes 2. Jog to mid-position 3. Command 1" move 4. Capture in Halscope 5. Analyze: Overshoot, settling time, following error

Acceptance Criteria: - Overshoot: < 10% - Settling time: < 200 ms - Following error (during motion): < 0.002" - Steady-state error: < 0.0001"

Circular Interpolation Test

G-Code (circular path):

```
G0 X2 Y0
G1 Z-0.1 F10
G2 I-2 J0 F100 ; Full circle, radius = 2"
G0 Z0.1
```

Measure: - Actual radius at multiple points (dial indicator or CMM) - Deviation from nominal = contouring error

Typical Results: - Well-tuned system: +/-0.001-0.002" radial error - Poor tuning: +/-0.005-0.010" radial error

Practical Cutting Test

Test Part: Simple square with rounded corners

G-Code:

```
G64 P0.005
G0 X0 Y0 Z0.1
G1 Z-0.1 F10
G1 X4 F100
G3 X4.5 Y0.5 I0 J0.5
G1 Y4
G3 X4 Y4.5 I-0.5 J0
G1 X0
G3 X-0.5 Y4 I0 J-0.5
G1 Y0
G3 X0 Y-0.5 I0.5 J0
G0 Z0.1
```


Measure: - Corner radius accuracy - Surface finish (visual and Ra measurement) - Dimensional accuracy (calipers/micrometer)

Summary

LinuxCNC provides powerful, flexible servo control:

Key Components: 1. **HAL:** Flexible hardware abstraction (connect anything to anything) 2. **PID Component:** Built-in position loops with feedforward 3. **Halscope:** Real-time oscilloscope for tuning 4. **Trajectory Planner:** Real-time look-ahead with blending

Tuning Process: 1. Configure INI file (axes, limits, scales) 2. Set up HAL (PID, encoders, motor drivers) 3. Tune with Halscope (P, FF1, D, I, FF2) 4. Test and validate (step response, circular interp)

Advanced Features: - Notch filters (resonance suppression) - Gantry kinematics (automatic synchronization) - Encoder index homing (high precision) - Real-time performance tuning

Next: 19.11 Implementation in Mach4

Next: 19.11 Implementation in Mach4

19.11 Implementation in Mach4

Mach4 Overview

Mach4: Commercial CNC control software for Windows, successor to Mach3.

Key Features: - Plugin-based motion control (external motion controller required) - Lua scripting for customization - Modern GUI with touchscreen support - Support for steppers and servos - Real-time trajectory planning (depends on plugin) - Cost: ~\$200 (hobby license)

Architecture:

```
G-Code -> Mach4 Core -> Motion Plugin -> External Controller
                                     (ESS, CSMIO, etc.)
                                     down
                                     Motor Drivers
                                     up
                                     Encoders
```

Key Difference from LinuxCNC: Mach4 relies on external motion controller hardware (not software-based real-time control).

Motion Controller Hardware

Common Motion Controllers

Ethernet SmoothStepper (ESS): - Ethernet connection to PC - 6-axis control - Step/direction output (steppers or step-servo drives) - Servo tuning parameters accessible via Mach4 - Cost: ~\$200-250

CNC4PC C11G (Galil-based): - USB connection - 4-axis control - Analog +/-10V outputs (servo drives) - Built-in PID loops (tuning via Mach4) - Cost: ~\$300-400

CSMIO/IP-A: - Ethernet connection - Up to 6 axes - Analog +/-10V outputs - Advanced features (probe, spindle control) - Cost: ~\$400-500

Smoothieboard: - Open-source firmware - USB connection - Step/direction outputs - Limited servo support - Cost: ~\$150-200

Selecting a Motion Controller

Stepper Systems: - ESS (best value) - Smoothieboard (open-source) - UC100 (USB, budget option)

Servo Systems: - CSMIO/IP-A (full-featured) - Galil-based cards (industrial-grade) - ESS (if using step-servo drives)

Key Consideration: Servo tuning accessibility (can you adjust PID gains from Mach4?)

Initial Setup and Configuration

Mach4 Installation

Steps: 1. Download Mach4 installer from machsupport.com 2. Install Mach4 (default path: C:/Mach4Hobby) 3. Install motion controller plugin (ESS, CSMIO, etc.) 4. Launch Mach4, select profile

Profile: Contains machine-specific configuration - Create new profile for your machine - Name: e.g., "MyMill", "RouterCNC"

Machine Configuration Wizard

Mach4 Menu: Configure -> Mach

Steps:

- 1. Units and Axis:** - Units: inch or mm - Active axes: X, Y, Z (select which axes present) - Homing: Enable if home switches installed
- 2. Motor Configuration** (per axis): - Steps per unit: (encoder resolution or step/dir ratio) - Velocity: Maximum velocity (in/min or mm/min) - Acceleration: Maximum acceleration (in/s² or mm/s²) - Motor direction: CW/CCW (invert if needed)
- 3. Soft Limits:** - Minimum position: -0.1 (slight negative for homing) - Maximum position: 24.0 (axis travel)
- 4. Homing:** - Home switch location (min or max end) - Home speed: Fast search (50 IPM) and slow latch (5 IPM) - Home offset: Distance from switch to zero position
- 5. Save Configuration**

Servo Tuning in Mach4

Accessing Servo Parameters

Method 1: Motor Configuration Screen: - Configure -> Mach -> Motor - Select axis - Servo tuning tab

Method 2: Registers (CSMIO/IP): - Configure -> Plugins -> CSMIO/IP-A - Servo parameters

Available Parameters (depends on controller): - P Gain (Proportional) - I Gain (Integral) - D Gain (Derivative) - Velocity Feedforward (if supported) - Acceleration Feedforward (if supported) - Maximum output - Deadband

Tuning Procedure (Mach4)

Example: ESS with servo drives

Step 1: Set Initial Gains - Open Motor Configuration -> Servo Tuning - Set conservative values: - P = 50 - I = 0 - D = 0 - Max Output = 100%

Step 2: Test Motion - Jog axis slowly (10 IPM) - Observe: Does axis follow smoothly? - If sluggish: Increase P - If oscillates: Reduce P

Step 3: Increase P Gain - Increment P by 25-50% - Test jog after each change - Continue until: - Slight overshoot appears, or - Oscillation begins - Back off 20-30% from oscillation point

Example: - P = 50: Sluggish - P = 100: Better, still slow - P = 150: Responsive - P = 200: Slight overshoot (8%) - P = 250: Oscillation - **Final:** P = 180-200

Step 4: Add Integral Gain - Set I = P / 20 (starting point) - Test: Does axis hold position when stopped? - Increase I until steady-state error eliminated - Watch for overshoot increase

Example: - I = 0: Position drifts 0.002" when stopped - I = 5: Drift = 0.0005" - I = 10: Drift < 0.0001" (good) - I = 20: Overshoot increases to 15% (too much) - **Final:** I = 10-12

Step 5: Add Derivative Gain - Set D = P / 10 (starting point) - Test: Does overshoot reduce? - Increase D until overshoot < 5-10% - Watch for jittery motion (noise amplification)

Example: - D = 0: Overshoot = 8% - D = 10: Overshoot = 5% - D = 20: Overshoot = 2%, slight jitter - **Final:** D = 12-15

Step 6: Test at Various Speeds - Jog slowly (10 IPM) - Jog medium (100 IPM) - Jog fast (max velocity) - Run test G-code program - Adjust gains if instability at specific speeds

Velocity Feedforward (if supported)

Check: Does your motion controller support velocity feedforward? - ESS: Limited support (check documentation) - CSMIO/IP: Yes (parameter available) - Galil-based: Yes

Configuration: - Set FF (or FF1) = 0.9 (starting point) - Jog at constant velocity - Measure following error (if controller displays) - Increase FF until following error minimized - Typical: FF = 0.95-1.0

Example (CSMIO/IP): - Parameter: “Velocity Feedforward Gain” - Range: 0-100% - Set to 95% for best tracking

External Drive Tuning

Some Servo Drives Have Internal PID Loops: - Yaskawa, Delta, Panasonic drives - Drive has velocity/current loops (fast, 10-20 kHz) - Mach4 only sends velocity or position commands

Configuration: - Tune drive first (use drive’s auto-tune if available) - Then tune Mach4 position loop (if applicable) - Cascaded loops: Drive (inner) + Mach4 (outer)

Example (Delta ASDA-A2): - Set drive to Position Mode or Velocity Mode - Run drive auto-tune (Parameter Pn102 = 2) - Drive tunes internal velocity loop - Mach4 sends position commands (no PID tuning needed in Mach4)

Trajectory Planning Configuration

Trajectory Parameters

Mach4 Menu: Configure -> Mach -> Trajectory

Parameters: - **Look Ahead:** Number of blocks to read ahead (50-200) - **CV Distance Mode:** Constant Velocity distance (blending mode) - **CV Tolerance:** Maximum path deviation (similar to G64 P) - **Stop on Angles >:** Angle threshold for exact stop (e.g., 90 deg)

Example Configuration: - Look Ahead: 100 blocks - CV Tolerance: 0.005” (finishing) or 0.020” (roughing) - Stop on Angles > 90 deg: Stop only at corners sharper than 90 deg

G-Code Blending Modes

G64: Constant Velocity (CV) mode - blend corners

Example:

G64

G1 X10 Y0 F100

G1 X10 Y10 ; Blend through (10,0) corner

G1 X0 Y10 ; Blend through (10,10) corner

Effect: Smooth continuous motion, corners rounded within CV tolerance.

G61: Exact Stop mode

G61

G1 X10 Y0 F100

G1 X10 Y10 ; Stop completely at (10,0)

Effect: Stop at every programmed point.

G61.1: Exact Path mode (maintain path, but don’t stop)

G61.1

G1 X10 Y0 F100

G1 X10 Y10 ; Follow path exactly, slow if needed

Effect: Maintains geometric accuracy, slows at corners to stay on path.

Motion Plugin Settings

ESS Plugin: - Configure -> Plugins -> ESS - Charge Pump: Enable (safety feature) - Step Resolution: 25 kHz typical - Encoder Resolution: Set if using encoders

CSMIO Plugin: - Configure -> Plugins -> CSMIO/IP-A - Encoder Setup: Input scaling, filtering - Analog Outputs: Range (+/-10V), offset - MPG Setup: Manual pulse generator

Homing Configuration

Homing Setup

Configure -> Mach -> Homing:

Per-Axis Parameters: - **Home Offset:** Distance from home switch to machine zero - **Home Speed %:** Percentage of max velocity for homing - **Slow Home Speed %:** Percentage for slow latch (after switch found) - **Home Direction:** + or - (which direction to search)

Example (X-Axis): - Home switch at X = -1" (left end) - Machine zero at X = 0" - Home Offset = 1.0" - Home Speed % = 50% (of max velocity) - Slow Home Speed % = 10% - Home Direction = Negative (search left)

Homing Sequence: 1. Press "Ref All Home" button 2. X-axis moves left at 50% speed 3. Triggers home switch 4. Backs off, approaches at 10% speed 5. Latches on switch 6. Moves to home offset position (0.0) 7. Repeat for Y, Z axes

Home Switch Wiring

Single Switch per Axis (simple): - Wire switch between input pin and ground - Normally Open (NO) or Normally Closed (NC) - Configure in Mach4: Input Signals -> Home X

Shared Home/Limit Switch: - One switch acts as both home and limit - Reduce wiring - Configure: Enable "Home is Limit" option

Index Pulse Homing (high precision): - Use encoder index pulse as latch - Requires motion controller with index support (CSMIO) - Repeatability: +/-1 encoder count

Testing and Diagnostics

Diagnostics Screen

Mach4 Menu: Diagnostics

Monitor: - **Axis Position:** Commanded vs. actual (if encoder feedback) - **Following Error:** Real-time display - **Inputs:** Home switches, limit switches, E-stop - **Outputs:** Motor enable, coolant, spindle

Test Motion: - Jog each axis - Observe diagnostics for: - Position tracking (actual follows commanded) - Following error magnitude - Smooth motion (no jumps or stuttering)

Circular Interpolation Test

G-Code:

```
G0 X2 Y0
G1 Z-0.1 F20
G2 I-2 J0 F100 ; 360 deg circle, R=2"
G0 Z0.1
M30
```

Run Program: - Observe motion (smooth circle, no faceting) - Measure actual radius (dial indicator or probe) - Compare to nominal (2.000")

Typical Results: - Good tuning: +/-0.002" radial error - Poor tuning: +/-0.010" radial error - Mechanical issues: Oval shape (different X/Y stiffness)

Velocity Override Test

Function: Test feed rate override (real-time speed adjustment)

Procedure: 1. Run G-code program at F100 2. During motion, adjust Feed Rate Override slider (50%-150%) 3. Observe: Smooth speed changes, no stuttering

Expected: Motion smoothly ramps to new feedrate.

Problem: Jerky motion or stuttering -> motion controller buffer issue or insufficient look-ahead.

Lua Scripting for Customization

Lua Basics in Mach4

Mach4 uses Lua for: - Custom M-codes (M100-M199) - Screen scripts (button actions) - PLC script (background logic, runs continuously) - Macros (reusable functions)

Accessing Lua Editor: - Operator -> Edit Screen - Right-click button -> Edit Script

Example: Custom Probing Routine

M-Code Script (m6.mcs - tool change macro):

```
-- M6 Tool Change with Probing
function m6()
    local selectedTool = mc.mcToolGetSelected(inst)
    local currentTool = mc.mcToolGetCurrent(inst)

    if (selectedTool == currentTool) then
        return -- No change needed
    end

    -- Move to tool change position
    mc.mcCntlGcodeExecuteWait(inst, "G53 G0 Z0")
    mc.mcCntlGcodeExecuteWait(inst, "G53 G0 X-1 Y-1")

    -- Prompt for manual tool change
    wx.wxMessageBox("Change to Tool #" .. selectedTool)

    -- Probe new tool length
    mc.mcCntlGcodeExecuteWait(inst, "G30") -- Move to probe position
    mc.mcCntlGcodeExecuteWait(inst, "G38.2 Z-2 F5") -- Probe down
    local probeZ = mc.mcAxisGetPos(inst, 2) -- Get Z position

    -- Set tool offset
    mc.mcToolSetData(inst, mc.MTOOL_MILL_HEIGHT, selectedTool, probeZ)
    mc.mcToolSetCurrent(inst, selectedTool)
end

if (mc.mcInEditor() == 1) then
```

```
m6()  
end
```

Use: Automatic tool length probing after tool change.

PLC Script (Background Tasks)

PLC Script Runs Continuously (every cycle):

Example: Monitor spindle speed, adjust feedrate

```
-- PLC Script: Adaptive Feed  
function PLCScript()  
    local spindleRPM = mc.mcSpindleGetCurrentRPM(inst)  
    local targetRPM = mc.mcSpindleGetCommandedRPM(inst)  
  
    -- If spindle load high (RPM drops), reduce feedrate  
    if (spindleRPM < targetRPM * 0.8) then  
        mc.mcCntlSetFRO(inst, 70) -- Reduce to 70% feedrate  
    else  
        mc.mcCntlSetFRO(inst, 100) -- Restore 100%  
    end  
end
```

Use: Prevent spindle stall during heavy cuts.

Plugin Development (Advanced)

Mach4 Plugin API

For Advanced Users: Create custom motion control plugins.

Language: C/C++

API: Mach4 Plugin SDK (available from machsupport.com)

Use Cases: - Custom motion controller hardware - Specialized kinematics (e.g., SCARA robot) - Integration with external systems

Example Plugins: - ESS Plugin (Ethernet SmoothStepper) - CSMIO Plugin (CS-Lab motion controllers) - Galil Plugin (Galil motion controllers)

Complexity: High (requires C++ programming, real-time considerations)

Common Issues and Solutions

Issue: Jerky Motion

Possible Causes: 1. Insufficient look-ahead buffer 2. PC performance (Mach4 running slow) 3. USB latency (if using USB controller) 4. Short G-code line segments (CAM output)

Solutions: - Increase look-ahead: Configure -> Mach -> Trajectory -> Look Ahead = 200 - Close background programs (browser, etc.) - Use Ethernet controller instead of USB - Adjust CAM tolerance (longer line segments)

Issue: Following Error

Symptom: Axis lags command, following error alarm

Causes: 1. P gain too low 2. No velocity feedforward 3. Mechanical binding (friction) 4. Motor undersized

Solutions: - Increase P gain (test for stability) - Enable velocity feedforward (if supported) - Lubricate ways, check for binding - Upgrade motor (more torque)

Issue: Oscillation

Symptom: Axis shakes or buzzes, unstable

Causes: 1. P or D gain too high 2. Mechanical resonance 3. Encoder noise 4. Loose coupling

Solutions: - Reduce P and D gains 20-30% - Add mechanical damping - Check encoder wiring, shielding - Tighten couplings, check alignment

Summary

Mach4 provides accessible servo control for hobby and professional CNC:

Key Components: 1. **External Motion Controller:** ESS, CSMIO, Galil (handles real-time control) 2. **Mach4 Software:** G-code interpretation, trajectory planning, GUI 3. **Lua Scripting:** Customization, macros, PLC logic

Tuning Process: 1. Configure machine (axes, limits, motors) 2. Set motion controller parameters (steps/unit, velocity, accel) 3. Tune servo gains (P, I, D, feedforward) 4. Test and validate (jog, circular interp, cutting)

Advantages: - Easy setup (GUI-based) - Good plugin support (motion controllers) - Lua scripting (flexible customization) - Active community support

Limitations: - Requires external motion controller (added cost) - Real-time performance depends on controller hardware - Less transparent than LinuxCNC (closed-source core)

Next: 19.12 Troubleshooting and Optimization

Next: 19.12 Troubleshooting and Optimization

19.12 Troubleshooting and Optimization

Systematic Troubleshooting Approach

Effective Troubleshooting Requires: 1. **Observation:** What exactly is the problem? When does it occur? 2. **Hypothesis:** What could cause this behavior? 3. **Testing:** Change one variable, observe result 4. **Verification:** Confirm root cause, not just symptom 5. **Documentation:** Record findings for future reference

Golden Rule: Change only one thing at a time.

Common Problems and Solutions

Problem 1: Oscillation (Instability)

Symptoms: - Axis shakes or vibrates - Audible buzzing or whining - Visible oscillation in position plot - May occur at rest or during motion

Observation Checklist: - Frequency of oscillation? (low ~1-5 Hz, mid ~10-50 Hz, high >100 Hz) - Occurs at rest, during motion, or both? - All axes or specific axis? - Consistent or intermittent?

Root Cause 1: Gains Too High **Test:** - Reduce P gain by 30-50% - If oscillation stops: Gains too high

Solution: - Reduce P gain to 70-80% of oscillation threshold - Add D gain to increase damping - Re-test for stability

Example: - P = 150: Oscillation at 15 Hz - Reduce to P = 100: Stable - Add D = 10: Can increase P to 120 while maintaining stability - **Final:** P = 120, D = 10

Root Cause 2: Mechanical Resonance **Test:** - Tap axis with hammer, observe ring-down - Use accelerometer + FFT (if available) - Increase gains slowly - oscillation appears at specific P value

Identify Resonance: - Tap test shows ring-down at ~200 Hz - As P increases, oscillation appears at 200 Hz - **Conclusion:** Exciting mechanical resonance

Solution: - **Option 1:** Add notch filter at resonance frequency - LinuxCNC: `loadrt notch`, set freq = 200 Hz - Eliminates resonance from control loop - **Option 2:** Mechanical damping - Add foam, rubber mounts, constrained-layer damping - Reduces resonance amplitude - **Option 3:** Lower gains - Avoid exciting resonance (accept slower response)

Notch Filter Configuration (LinuxCNC):

```
loadrt notch names=notch.x
setp notch.x.freq 200
setp notch.x.q 5.0
addf notch.x servo-thread
net x-pid-out pid.x.output => notch.x.in
net xoutput notch.x.out => pwmgen.0.value
```

Root Cause 3: Encoder Noise **Test:** - Observe encoder position with axis stationary - If position jitters (+/-several counts): Noise problem

Sources: - Electrical interference (motor cables near encoder cables) - Poor shielding - Grounding issues - Encoder resolution too coarse for application

Solution: - **Shielding:** Use shielded twisted-pair cables for encoders - **Separation:** Route encoder cables away from motor power cables (6" minimum) - **Grounding:** Ground shield at one end only (prevent ground loops) - **Filtering:** Add low-pass filter to encoder signals (hardware or software) - **Encoder upgrade:** Higher resolution encoder (reduces quantization noise)

Software Filter (LinuxCNC):

```
loadrt lowpass names=lowpass.x-pos
setp lowpass.x-pos.gain 1.0
setp lowpass.x-pos.time-constant 0.001 # 1 ms filter
addf lowpass.x-pos servo-thread
net x-pos-raw encoder.0.position => lowpass.x-pos.in
net x-pos-filt lowpass.x-pos.out => pid.x.feedback
```

Root Cause 4: Loose Coupling or Bearing Test: - Manually move axis (motors disabled) - Feel for play, looseness, binding - Check coupling screws, bearing preload

Solution: - Tighten coupling set screws - Adjust bearing preload - Replace worn components

Problem 2: Following Error Alarm

Symptoms: - Controller stops with “Following Error” alarm - Error message: “Joint X following error exceeded limit” - Occurs during rapid moves or cutting

Observation: - When does it occur? (rapids, heavy cuts, specific moves?) - Following error magnitude at fault? - Consistent location or random?

Root Cause 1: P Gain Too Low Test: - Observe following error during motion (before alarm) - If error consistently approaches limit: Insufficient gain

Solution: - Increase P gain by 25-50% - Verify stability (no oscillation) - Increase following error limit if necessary (temporary)

Example: - Following error limit: 0.050” - Actual error during rapids: 0.045” (close to limit) - Increase P from 80 to 120 - New following error: 0.020” (safe margin)

Root Cause 2: No Velocity Feedforward Test: - Observe following error during constant velocity motion - Large error during cruise phase: Missing feedforward

Solution: - Add velocity feedforward (FF1) - Start with $FF1 = 0.9$, increase to 1.0 - Following error should reduce 5-10x

Example: - Without FF1: Following error = 0.008” during 200 IPM motion - With $FF1 = 0.95$: Following error = 0.0008” - **10x improvement**

Root Cause 3: Mechanical Obstruction Test: - Following error occurs at specific location - Axis physically encounters resistance

Check: - Binding (misaligned bearings, rails) - Crash damage - Chips/debris in ways - Insufficient lubrication

Solution: - Inspect mechanical system - Clean ways, screws - Realign components - Lubricate

Root Cause 4: Motor Undersized Test: - Following error increases under load (cutting forces) - Motor reaches torque limit

Calculation:

$$\text{Required Torque} = (\text{Inertia} \times \text{Accel}) + \text{Friction} + \text{Load}$$

Example: - Max acceleration: 200 in/s² - Axis inertia (reflected): 0.01 kgm² - *Required torque: $0.01 \times (200 \times 0.0254) = 0.051$ Nm* + friction + load - If motor continuous torque < required: Undersized

Solution: - Upgrade to higher torque motor - Reduce acceleration (temporary) - Reduce cutting forces (slower feedrate, lighter DOC)

Problem 3: Poor Surface Finish

Symptoms: - Visible vibration marks (chatter) - Ripples or waves in surface - Inconsistent finish

Observation: - Pattern regular or random? - Occurs on all surfaces or specific directions? - Frequency of pattern? (measure wavelength)

Root Cause 1: Mechanical Vibration **Test:** - Accelerometer on spindle or toolholder - Measure vibration frequency - Match to surface finish wavelength

Calculation:

$$\text{Wavelength} = \frac{\text{Feedrate}}{\text{Vibration Frequency}}$$

Example: - Feedrate: 100 IPM = 1.67 in/s - Measured wavelength: 0.010" - Vibration frequency: $1.67 / 0.010 = 167$ Hz

Solution: - Identify vibration source (spindle imbalance, tool runout, resonance) - Add damping or stiffness - Adjust spindle speed (avoid resonant frequencies) - Use different feeds/speeds to shift frequency

Root Cause 2: Servo Tuning Issues **Test:** - Run circular interpolation test (perfect circle G-code) - Measure actual path deviation (contouring error) - Large error (>0.005"): Tuning problem

Typical Issues: - Following error during motion - Axis response mismatch (X fast, Y slow -> oval circles) - Inadequate feedforward

Solution: - Balance servo response (match bandwidth of all axes) - Increase P gain (reduce following error) - Add velocity and acceleration feedforward - Verify circular path within tolerance

Root Cause 3: Excessive Corner Blending **Test:** - Measure corner radius vs. programmed - Large radius (>tolerance): Excessive blending

G-Code Check:

G64 P0.020 ; Tolerance too loose for finishing

Solution: - Reduce blend tolerance for finishing passes

G64 P0.002 ; Tighter tolerance

- Trade-off: Longer cycle time, better finish
- Use G61 (exact stop) for critical corners

Problem 4: Inconsistent Positioning

Symptoms: - Parts dimensionally inconsistent - Position varies between runs - Hysteresis (different position depending on approach direction)

Observation: - Repeatability test: Move to position 10x, measure variation - Unidirectional vs. bidirectional error

Root Cause 1: Backlash **Test:** - Move axis +1", then -1", measure actual position - If position offset = backlash amount

Measurement: - Mount dial indicator on spindle - Touch off on fixed surface - Move +0.5", return to zero: read indicator - Difference = backlash

Solution: - **Mechanical:** Anti-backlash nut, preload - Replace worn ballscrew nut - Adjust preload (eliminate play) - **Software compensation** (temporary): - LinuxCNC: Set backlash parameter - Mach4: Set backlash compensation - Limitations: Only compensates slow moves

LinuxCNC Backlash Compensation:

[**AXIS_X**]

BACKLASH = 0.002 ; 0.002" backlash compensation

Better Solution: Eliminate backlash mechanically (more accurate).

Root Cause 2: Thermal Expansion **Test:** - Measure position when cold, after warm-up (30-60 min) - Position shift indicates thermal growth

Typical Expansion: - Aluminum: 13 $\mu\text{m}/\text{m}/\text{degC}$ (13 ppm/ degC) - Steel: 11 $\mu\text{m}/\text{m}/\text{degC}$ - Example: 1 meter (40") aluminum, 10 degC rise -> 130 μm (0.005") growth

Solution: - Allow warm-up before precision work - Temperature-controlled environment - Thermal compensation (measure, correct in software) - Use low-expansion materials (granite, carbon fiber, Invar)

Root Cause 3: Missed Steps (Steppers) **Test** (stepper systems): - Command large move (e.g., 10") - Measure actual distance - If short: Missed steps

Causes: - Motor undersized (insufficient torque) - Acceleration too high (resonance) - Mechanical binding

Solution: - Reduce acceleration (avoid resonance) - Increase motor current (if within rating) - Upgrade to larger motor - **Best:** Convert to closed-loop (servos detect/correct errors)

Problem 5: Noise and Jitter

Symptoms: - High-frequency jitter in motion - Audible whine from motors - "Nervous" behavior

Observation: - Frequency of jitter? (low hum vs. high whine) - Occurs at rest, during motion, or both?

Root Cause 1: Derivative Gain Too High **Test:** - Reduce D gain by 50% - If jitter reduces: Derivative amplifying noise

Solution: - Reduce D gain to acceptable level - Add low-pass filter to derivative term - Improve encoder resolution (reduce quantization noise)

Root Cause 2: Quantization Noise (Coarse Encoder) **Calculation:**

$$\text{Resolution} = \frac{\text{Screw Pitch}}{\text{Encoder CPR} \times 4}$$

Example: - Encoder: 500 CPR -> 2000 counts/rev (4x quadrature) - Screw pitch: 0.2 in/rev - Resolution: $0.2 / 2000 = 0.0001$ " per count

If D gain high, +/-1 count jitter -> large velocity estimate change -> jittery motor command.

Solution: - Upgrade encoder (2000 CPR -> 10,000 CPR: 5x better resolution) - Filter encoder signal (software low-pass) - Reduce D gain

Root Cause 3: Ground Loops **Test:** - Disconnect encoder shield ground at one end - If noise reduces: Ground loop problem

Ground Loop: - Shield grounded at both ends - Current flows through shield (creates magnetic field) - Couples noise into signal wires

Solution: - Ground shield at drive/controller end only (not motor end) - Use isolated encoder power supply - Check for multiple ground paths (eliminate)

Performance Optimization

Maximizing Cycle Time Reduction

Goal: Minimize total machining time while maintaining quality.

Strategies:

- 1. Optimize Trajectory Planning:** - Use blending mode (G64) instead of exact stop (G61) - Set appropriate tolerance (G64 P0.005 typical) - Look-ahead buffer size: 100-200 blocks
- 2. Increase Velocities and Accelerations** (if mechanically sound): - Gradually increase max velocity (test for vibration, accuracy) - Increase acceleration (test for following error, overshoot) - Typical gains: 20-50% improvement before reaching limits
- 3. Add Feedforward Control:** - Velocity feedforward (FF1): Reduces following error 5-10x - Acceleration feedforward (FF2): Reduces transient errors - Allows faster motion with same accuracy
- 4. CAM Optimization:** - Larger line segment tolerance (fewer short segments) - Avoid unnecessary Z retracts - Optimize tool paths (minimize rapid moves) - Trochoidal milling (constant engagement, higher feedrates)
- 5. Adaptive Feed:** - Monitor spindle load (current or RPM drop) - Reduce feedrate if load high (prevent tool breakage) - Increase feedrate if load low (maximize metal removal rate)

Improving Contouring Accuracy

Goal: Minimize path deviation during multi-axis motion.

Strategies:

- 1. Match Axis Response:** - Tune all axes to similar bandwidth (20-50 Hz typical) - Use circular interpolation test to verify - Adjust gains to eliminate oval circles
- 2. Increase Control Loop Rate:** - LinuxCNC: 1-2 kHz typical (limited by PC real-time performance) - Mach4 + external controller: 5-20 kHz (depends on controller) - Higher loop rate -> better tracking -> lower contouring error
- 3. Mechanical Improvements:** - Increase stiffness (bigger rails, thicker frame) - Reduce moving mass (lighter gantry, aluminum vs. steel) - Better bearings (preloaded, higher stiffness)
- 4. Use Linear Encoders** (glass scales): - Eliminate ballscrew pitch errors - Directly measure position - Typical improvement: 0.0005" -> 0.0001" accuracy

Noise Reduction

Goal: Quiet operation, reduce vibration.

Strategies:

- 1. Mechanical Damping:** - Foam padding under machine base - Constrained-layer damping (metal-rubber-metal sandwich) - Tuned mass dampers for specific resonances
- 2. S-Curve Profiles** (jerk-limited): - Replace trapezoidal with S-curve - Smoother acceleration transitions - Reduces excitation of mechanical resonances
- 3. Notch Filters:** - Identify resonance frequencies (tap test, frequency sweep) - Add notch filter at each resonance - Allows higher gains without exciting resonances
- 4. Quiet Motor Drives:** - Higher PWM frequency (20-40 kHz, inaudible) - Sinusoidal commutation (smoother than trapezoidal) - Current control tuning (reduce current ripple)

Benchmarking and Validation

Performance Metrics

- 1. Positioning Accuracy:** - Measure: Laser interferometer, length standards - Typical: ± 0.001 - 0.005 " (hobbyist), ± 0.0002 - 0.001 " (industrial)
- 2. Repeatability:** - Measure: Return to same position 10x, record variation - Typical: ± 0.0001 - 0.0005 " (hobbyist), ± 0.00005 - 0.0002 " (industrial)
- 3. Contouring Accuracy:** - Measure: Circular interpolation test, compare actual vs. nominal radius - Typical: ± 0.002 - 0.005 " (hobbyist), ± 0.0005 - 0.002 " (industrial)
- 4. Cycle Time:** - Measure: Time to complete test part - Compare: Before/after optimization - Typical improvement: 20-50% with tuning and trajectory optimization
- 5. Surface Finish:** - Measure: Surface roughness (Ra, Rz) with profilometer - Visual inspection (chatter marks, start/stop marks) - Typical: Ra = 50-200 μin (1.3-5 μm) for milling

Standard Test Programs

1. Circular Interpolation Test:

```
G0 X2 Y0 Z0.1
G1 Z-0.1 F20
G2 I-2 J0 F100 ; Full circle, R=2"
G0 Z0.1
M30
```

Measure: Radial deviation at 0 deg, 90 deg, 180 deg, 270 deg

2. Square with Rounded Corners:

```
G64 P0.005
G0 X0 Y0 Z0.1
G1 Z-0.05 F10
G1 X4 F100
G3 X4.5 Y0.5 I0 J0.5
G1 Y4
G3 X4 Y4.5 I-0.5 J0
G1 X0
G3 X-0.5 Y4 I0 J-0.5
G1 Y0
G3 X0 Y-0.5 I0.5 J0
G0 Z0.1
M30
```

Measure: Corner radius, dimensional accuracy, cycle time

- 3. Ballbar Test (ISO 230-4):** - Circular motion with telescoping ballbar - Measures geometric errors, servo performance - Professional tool (~\$5000-15000)

Documentation

Record Final Configuration:

- 1. Machine Specifications:** - Travel: X, Y, Z - Max velocity: per axis - Max acceleration: per axis - Motor specifications (model, torque, speed) - Encoder specifications (model, resolution) - Drive specifications (model, voltage, current)

2. Servo Tuning Parameters: | Axis | P | I | D | FF1 | FF2 | Bias | Max Output | |——|——|——|——|——|——|
 |——|——|——|——|——|——| | X | 125 | 10 | 15 | 0.95 | 0.002 | 0.0 | 10.0 | | Y | 120 | 10 | 14 | 0.94 | 0.002 | 0.0 | 10.0 | | Z
 | 100 | 12 | 12 | 0.90 | 0.001 | 2.5 | 10.0 |

3. Test Results: - Date of testing - Circular interpolation test: +/-0.0015" radial error - Repeatability: +/-0.0002" - Cycle time (test part): 3.2 minutes

4. Known Issues and Workarounds: - Resonance at 247 Hz (notch filter active) - Thermal drift: 0.001"/hour (allow 30 min warm-up)

Advanced Diagnostics

Frequency Response Measurement

Purpose: Measure actual system bandwidth, identify resonances.

Method (LinuxCNC with Halscope): 1. Apply sinusoidal position command (varying frequency) 2. Measure position response amplitude and phase 3. Plot Bode diagram (gain and phase vs. frequency) 4. Identify: Bandwidth, resonances, phase margin

Tools: - MATLAB/Octave: Signal processing toolbox - Python: scipy.signal - LinuxCNC: External script + Halscope data export

Example Findings: - Bandwidth: 35 Hz (-3 dB point) - Resonance at 247 Hz (+8 dB peak) - Phase margin: 42 deg (acceptable, 30-60 deg target)

Action: Add notch filter at 247 Hz, increase P gain (bandwidth -> 50 Hz).

Modal Analysis (Mechanical)

Purpose: Identify mechanical resonance modes (frequency, damping, mode shape).

Method: 1. Impact hammer + accelerometer 2. Measure frequency response function (FRF) 3. FFT analysis 4. Identify peaks (resonances)

Software: - Professional: ME'scope, LMS Test.Lab (high) - DIY: Smartphone accelerometer app + Python FFT

Example: - Mode 1: 120 Hz (gantry vertical bending) - Mode 2: 247 Hz (Z-axis torsion) - Mode 3: 380 Hz (spindle holder)

Action: Add bracing to reduce Mode 1, notch filter for Mode 2.

Summary

Systematic troubleshooting and optimization achieve high-performance CNC:

Key Practices: 1. **Systematic Approach:** Observe, hypothesize, test, verify 2. **One Variable at a Time:** Isolate root cause 3. **Documentation:** Record configuration, results 4. **Benchmarking:** Measure performance objectively

Common Issues: - **Oscillation:** Gains too high, mechanical resonance, noise - **Following Error:** Gains too low, no feedforward, mechanical resistance - **Poor Finish:** Vibration, tuning mismatch, excessive blending - **Inconsistent Position:** Backlash, thermal expansion, missed steps

Optimization Strategies: - Feedforward control (biggest performance gain) - Trajectory planning (blending, look-ahead) - Mechanical improvements (stiffness, damping) - Advanced techniques (notch filters, S-curves)

Validation: - Circular interpolation test (contouring accuracy) - Repeatability testing (precision) - Surface finish measurement (quality) - Cycle time comparison (throughput)

Final Thought: Advanced control transforms mechanical machines into precision instruments. Proper tuning and optimization unlock machine potential.

Module 19 Complete: You now have comprehensive knowledge of advanced control systems for CNC applications.

Continue Learning: - Practice tuning on real machines - Study control theory in depth (textbooks, courses)
- Experiment with advanced techniques (notch filters, adaptive control) - Join CNC communities (LinuxCNC forum, CNCzone, etc.)

Next Modules: - Module 20: CAM Software and Toolpath Generation - Module 21: Advanced Manufacturing Techniques

19.2 Control System Theory

Introduction to Control Theory

Control theory provides the mathematical foundation for understanding and designing feedback control systems. While you can tune PID controllers by trial and error, understanding control theory enables:

- **Predictable Results:** Anticipate system behavior before implementation
- **Systematic Design:** Calculate gains mathematically instead of guessing
- **Troubleshooting:** Diagnose problems by analyzing frequency response
- **Optimization:** Design controllers for specific performance criteria

Balance: This section provides enough theory to be useful without requiring advanced mathematics. Focus on concepts and practical application.

Transfer Functions

Definition

A **transfer function** describes the input-output relationship of a linear time-invariant (LTI) system in the frequency domain.

Mathematical Definition:

$$G(s) = \frac{Y(s)}{U(s)}$$

where: - s = complex frequency variable (Laplace domain) - $Y(s)$ = output (Laplace transform) - $U(s)$ = input (Laplace transform) - $G(s)$ = transfer function

Physical Meaning: How much output you get for a given input, as a function of frequency.

Laplace Transform Basics

The **Laplace transform** converts time-domain functions to frequency domain:

$$\mathcal{L}\{f(t)\} = F(s) = \int_0^{\infty} f(t)e^{-st} dt$$

Common Transforms: | Time Domain | Laplace Domain | |-----|-----| | $\delta(t)$ (impulse) | 1 | | $u(t)$ (step) | $\frac{1}{s}$ | | e^{-at} | $\frac{1}{s+a}$ | | $\sin(\omega t)$ | $\frac{\omega}{s^2+\omega^2}$ | | $\frac{df}{dt}$ (derivative) | $sF(s)$ | | $\int f dt$ (integral) | $\frac{F(s)}{s}$ |

Why Useful: Differential equations in time domain -> algebraic equations in Laplace domain (easier to solve).

Example: Time domain: $\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = u$

Laplace domain: $s^2X(s) + 2\zeta\omega_nsX(s) + \omega_n^2X(s) = U(s)$

Transfer function: $G(s) = \frac{X(s)}{U(s)} = \frac{1}{s^2 + 2\zeta\omega_ns + \omega_n^2}$

First-Order System

General Form:

$$G(s) = \frac{K}{\tau s + 1}$$

where: - K = DC gain (steady-state gain) - τ = time constant

Time Domain (Step Response):

$$y(t) = K(1 - e^{-t/\tau})$$

Characteristics: - **Rise time:** $t_r \approx 2.2\tau$ (10% to 90%) - **Settling time (2%):** $t_s \approx 4\tau$ - **Time constant** τ : Time to reach 63% of final value

Example: RC Low-Pass Filter - $V_{in}(s) \rightarrow G(s) \rightarrow V_{out}(s)$ - $G(s) = \frac{1}{RC \cdot s + 1}$ - Time constant: $\tau = RC$

CNC Example: Servo motor velocity response - Command step in voltage $\rightarrow$ motor accelerates to final velocity - $G(s) = \frac{K_t}{Js + b}$ approx first-order system - K_t = torque constant, J = inertia, b = viscous damping

Second-Order System

General Form:

$$G(s) = \frac{K\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

where: - K = DC gain - ω_n = natural frequency (rad/s) - ζ = damping ratio (dimensionless)

Physical Meaning: Mass-spring-damper system - Mass (m) $\rightarrow$ inertia - Spring (k) $\rightarrow$ stiffness - Damper (c) $\rightarrow$ friction/damping

$$\omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{c}{2\sqrt{km}}$$

CNC Interpretation: - Moving mass: gantry, table, spindle head - Spring: mechanical compliance (frame, screws, couplings) - Damper: friction, viscous damping in ways

Damping Ratio Effects

Underdamped ($\zeta < 1$): - Oscillatory response - Overshoot present - Settling time moderate

Critical Damping ($\zeta = 1$): - Fastest response without overshoot - No oscillation - Optimal for many applications

Overdamped ($\zeta > 1$): - Slow, sluggish response - No overshoot - Long settling time

Percent Overshoot:

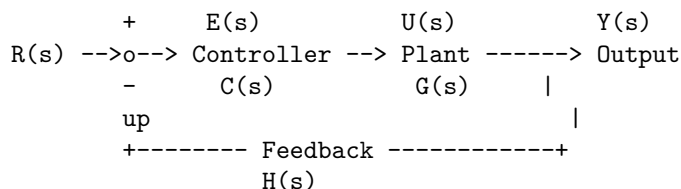
$$PO = e^{-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}} \times 100\%$$

Examples: | ζ | Overshoot | Application | |-----|-----|-----| | 0.3 | 37% | Too oscillatory for CNC | | 0.5 | 16% | Acceptable for rapids | | 0.707 | 4.3% | Common CNC target | | 0.9 | 0.2% | Precision positioning | | 1.0 | 0% | Critically damped |

Typical CNC Target: $\zeta = 0.7 - 0.9$ (slight underdamping, fast response, minimal overshoot)

Block Diagrams

Basic Feedback Loop:



Closed-Loop Transfer Function:

$$T(s) = \frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)H(s)}$$

Unity Feedback ($H(s) = 1$):

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}$$

Example: Proportional controller ($C(s) = K_P$), first-order plant ($G(s) = \frac{1}{\tau s + 1}$)

$$T(s) = \frac{K_P \cdot \frac{1}{\tau s + 1}}{1 + K_P \cdot \frac{1}{\tau s + 1}} = \frac{K_P}{\tau s + 1 + K_P}$$

Effect of Increasing K_P : - DC gain: $\frac{K_P}{1+K_P} \rightarrow 1$ as $K_P \rightarrow \infty$ - Time constant: $\frac{\tau}{1+K_P} \rightarrow 0$ (faster response)

Frequency Response

Definition

Frequency response: System output when input is sinusoid at various frequencies.

Sinusoidal Input: $u(t) = A \sin(\omega t)$

Steady-State Output: $y(t) = |G(j\omega)| \cdot A \sin(\omega t + \angle G(j\omega))$

where: - $|G(j\omega)|$ = magnitude (gain at frequency ω) - $\angle G(j\omega)$ = phase shift at frequency ω

Key Concept: Replace s with $j\omega$ in transfer function to get frequency response.

$$G(j\omega) = G(s) \Big|_{s=j\omega}$$

Bode Plots

Bode plot: Graphical representation of frequency response - **Magnitude plot:** $20 \log_{10} |G(j\omega)|$ (dB) vs. frequency (log scale) - **Phase plot:** $\angle G(j\omega)$ (degrees) vs. frequency (log scale)

Why Bode Plots: - Visualize system response across frequency spectrum - Identify bandwidth, resonances, phase lag - Design and analyze controllers - Assess stability margins

Bandwidth

Definition: Frequency at which closed-loop gain drops to -3 dB below DC value.

$$|T(j\omega_{BW})| = \frac{|T(j0)|}{\sqrt{2}} = 0.707 \times |T(j0)|$$

Physical Meaning: Maximum frequency of input commands system can follow accurately.

Example: - System bandwidth: 50 Hz - Sinusoidal position command at 10 Hz: System follows accurately (<3 dB attenuation) - Sinusoidal position command at 100 Hz: System cannot follow (>10 dB attenuation)

CNC Context: - Higher bandwidth -> can follow rapid direction changes (corners, curves) - Typical CNC servo bandwidth: 20-100 Hz - High-performance systems: 100-200 Hz

Gain Margin and Phase Margin

Stability Margins: Measure of “how far” from instability the closed-loop system is.

Gain Margin (GM): - Additional gain that can be added before system becomes unstable - Measured at frequency where phase = -180 deg (phase crossover frequency)

$$GM = -20 \log_{10} |L(j\omega_{pc})|_{dB} \text{ at } \angle L(j\omega_{pc}) = -180deg$$

Phase Margin (PM): - Additional phase lag system can tolerate before instability - Measured at frequency where magnitude = 0 dB (gain crossover frequency)

$$PM = 180deg + \angle L(j\omega_{gc}) \text{ at } |L(j\omega_{gc})|_{dB} = 0$$

Typical Specifications: - **Gain Margin:** >6 dB (factor of 2) - **Phase Margin:** 30-60 deg (45 deg common target)

Phase Margin vs Damping Ratio (approximation):

$$\zeta \approx \frac{PM}{100}$$

Examples: - PM = 30 deg -> $\zeta \approx 0.3$ (37% overshoot) - PM = 45 deg -> $\zeta \approx 0.45$ (20% overshoot) - PM = 60 deg -> $\zeta \approx 0.6$ (10% overshoot)

Summary

Control system theory provides the foundation for systematic servo tuning:

Key Concepts: 1. **Transfer functions:** Mathematical models of system dynamics 2. **Frequency response:** How system responds to different frequencies 3. **Bode plots:** Visualize gain and phase vs. frequency 4. **Stability margins:** Quantify “distance” from instability

Practical Application: - Higher K_P : Increases bandwidth, reduces phase margin - Add K_I : Eliminates steady-state error, reduces phase margin - Add K_D : Increases phase margin, extends bandwidth

Next Steps: - Apply theory to PID tuning methods (Section 19.4) - Learn systematic tuning procedures - Implement in real systems (Sections 19.10-19.11)

Next: 19.3 PID Control Fundamentals

19.3 PID Control Fundamentals

PID Overview

Proportional-Integral-Derivative (PID) control is the most widely used feedback control algorithm in industrial applications, including CNC servo systems.

Why PID Dominates: - Simple to understand conceptually - Only three parameters to tune - Effective for most single-input, single-output systems - Decades of industrial experience and tuning methods - Implemented in virtually all servo drives and CNC controllers

The Error Signal

Position Error Definition

$$e(t) = r(t) - y(t)$$

where: - $e(t)$ = error at time t - $r(t)$ = reference (commanded position) - $y(t)$ = measured position (actual)

Example: - Commanded position: 5.0000" - Actual position: 4.9985" - Error: 5.0000 - 4.9985 = +0.0015" (positive = lagging)

Sign Convention: - **Positive error:** Actual position lags command (need to speed up) - **Negative error:** Actual position leads command (need to slow down)

Continuous vs Discrete Time

Continuous-Time PID (theoretical):

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt}$$

Discrete-Time PID (actual implementation in digital controller):

$$u[k] = K_P e[k] + K_I \sum_{i=0}^k e[i] \Delta t + K_D \frac{e[k] - e[k-1]}{\Delta t}$$

where: - k = sample index ($k, k-1, k-2, \dots$) - Δt = sample period (e.g., 0.001 s for 1 kHz loop)

Example: 1 kHz position loop: $\Delta t = 0.001$ s - Sample 0: $t = 0.000$ s - Sample 1: $t = 0.001$ s - Sample 2: $t = 0.002$ s - ...

Proportional (P) Term

Theory

Formula:

$$u_P(t) = K_P \cdot e(t)$$

Physical Meaning: Spring connecting actual position to commanded position - Larger error $\rightarrow$ larger restoring force - Spring stiffness = K_P

Effect on System: - Higher $K_P \rightarrow$ stiffer position control - Higher $K_P \rightarrow$ faster response - Higher $K_P \rightarrow$ less steady-state error (but never zero with P alone)

Steady-State Error with P-Only Control

Problem: Proportional control alone cannot eliminate steady-state error.

Why: At steady-state, error must be non-zero to produce motor command.

Example: - P-only controller: $K_P = 100$ (units: command per inch of error) - Motor requires 50 units to overcome friction and hold position - At steady-state: $50 = 100 \times e_{ss}$ - Therefore: $e_{ss} = 0.50 / 100 = 0.005$ " (persistent error)

Doubling K_P to 200: - $e_{ss} = 50 / 200 = 0.0025$ " (half the error)

Key Insight: Higher K_P reduces but doesn't eliminate steady-state error.

Proportional Gain Units

Typical Units: - **Command per distance:** volts per inch, % per mm - **Dimensionless** (if command and error in same units) - **In some systems:** Force per distance (N/mm) - acts like spring constant

LinuxCNC Example: - $K_P = 100$ (units: per second) - With 1 kHz position loop - Effective gain = $100 / 1000 = 0.1$ (10% correction per ms per inch error)

Example Calculation: - Error: 0.010" - $K_P = 100$ - Proportional output: $100 \times 0.010 = 1.0$ (velocity command, inches/second)

Controller commands 1.0 in/s velocity to reduce error.

Practical Proportional Tuning

Start Low: - Begin with $K_P = 10$ -20 - Verify stable (no oscillation)

Increase Gradually: - Increase by 20-50% each iteration - Test step response after each increase - Look for overshoot or oscillation

Optimal K_P : - Just below onset of sustained oscillation - Typically results in 5-15% overshoot - Will be refined with derivative term

Example Progression: | K_P | Overshoot | Settling Time | Notes | |——-|———|—————|——-| | 10 | 0% | 500 ms | Too slow, overdamped | | 25 | 2% | 200 ms | Better, still slow | | 50 | 8% | 100 ms | Good response | | 100 | 25% | 80 ms | Too much overshoot | | 75 | 12% | 90 ms | **Target: Will add D term** |

Derivative (D) Term

Theory

Formula:

$$u_D(t) = K_D \cdot \frac{de(t)}{dt}$$

Physical Meaning: Damper (shock absorber) - Opposes rapid changes in error - Provides "braking" action as position approaches target

Effect on System: - Reduces overshoot (adds damping) - Improves stability - Allows higher K_P without oscillation - Faster settling time

Discrete Implementation

Simple Derivative:

$$\frac{de[k]}{dt} \approx \frac{e[k] - e[k-1]}{\Delta t}$$

Problem: Amplifies high-frequency noise

Example of Noise Amplification: - True error: 0.001" - Encoder noise: +/-0.00005" (typical) - Measured error jumps: 0.00095" -> 0.00105" (due to noise) - Derivative: (0.00105 - 0.00095) / 0.001 = 0.10 in/s (huge!)
 - With $K_D = 50$: Output = 50 x 0.10 = 5.0 (inappropriate response to noise)

Solution: Filter derivative term or use derivative-on-measurement.

Derivative-on-Measurement (Recommended)

Instead of differentiating error, differentiate measurement:

$$u_D(t) = -K_D \cdot \frac{dy(t)}{dt}$$

Advantage: Step changes in setpoint don't cause derivative kick

Example: - Command steps from 0" to 1.000" instantly - Error derivative: (1.000 - 0) / Deltat = huge spike! - Measurement derivative: smooth change as axis accelerates (no spike)

Most Modern Controllers: Use derivative-on-measurement by default.

Derivative Filtering

Low-Pass Filter on Derivative:

$$D_{filtered}(s) = \frac{K_D s}{1 + \tau_D s}$$

where τ_D = derivative filter time constant (typically Δt to $10\Delta t$)

Effect: - Reduces noise amplification - Slightly delays derivative action - More stable in presence of measurement noise

Practical Implementation: First-order digital filter

$$D_{filtered}[k] = \alpha D[k] + (1 - \alpha) D_{filtered}[k - 1]$$

where $\alpha = \frac{\Delta t}{\Delta t + \tau_D}$ (typical $\alpha = 0.1-0.3$)

Derivative Gain Tuning

Start with Zero: - Tune P term first - Note overshoot and settling time

Add Derivative Gradually: - Start with $K_D = K_P / 10$ - Increase until overshoot <5-10% - Typical ratio: $K_D = (0.05-0.20) \times K_P$

Example: - $K_P = 75$, overshoot = 12% - Add $K_D = 5$: overshoot = 8% - Add $K_D = 10$: overshoot = 4%
 - Add $K_D = 15$: overshoot = 2% (critically damped) - **Select** $K_D = 10-12$ for slight underdamping

Watch for Noise: - If system becomes "nervous" (jittery), reduce K_D - Add more filtering if needed

Integral (I) Term

Theory

Formula:

$$u_I(t) = K_I \int_0^t e(\tau) d\tau$$

Physical Meaning: Memory of past errors - Accumulates error over time - Generates command proportional to total accumulated error

Effect on System: - Eliminates steady-state error - Compensates for constant disturbances (friction, gravity, drag) - Can cause overshoot and oscillation if too high - Slows response if too high

Why Integral is Necessary

Constant Disturbance Example: - Vertical axis with gravity load: 50 lb constant force - P and D terms only: Axis sags until PD command = 50 lb - Steady-state error remains (position below target)

With Integral Term: - Error accumulates over time - Integral term ramps up - Eventually produces 50 lb command to balance gravity - Position error goes to zero

Discrete Implementation

Rectangular Integration:

$$I[k] = I[k - 1] + e[k] \cdot \Delta t$$

Trapezoidal Integration (more accurate):

$$I[k] = I[k - 1] + \frac{e[k] + e[k - 1]}{2} \cdot \Delta t$$

Example: - $\Delta t = 0.001$ s - Error sequence: 0.010", 0.008", 0.006", 0.004", 0.002" - Integral: 0, 0.010x0.001, 0.010x0.001+0.008x0.001, ... - After 5 samples: $I = 0.000030$ in*s

With $K_I = 1000$:

$$u_I = 1000 \times 0.000030 = 0.030$$

Integral Windup

Problem: Integral term accumulates to very large values during: - Startup (axis far from target) - Saturated motor (commanded torque exceeds motor capability) - Mechanical limit encountered

Result: - Huge integral term persists after error eliminated - Causes large overshoot - Slow recovery (must "unwind" integral term)

Example: - Commanded move: 0" -> 10" - During move: large error accumulates (integral = 10 in*s) - Arrive at target: error = 0, but integral term remains - Integral term pushes past target (overshoot) - Takes time for negative error to cancel accumulated integral

Anti-Windup Techniques

Method 1: Integral Clamping Limit maximum integral value:

```
if (integral > MAX_INTEGRAL) integral = MAX_INTEGRAL;
if (integral < MIN_INTEGRAL) integral = MIN_INTEGRAL;
```

Method 2: Conditional Integration Only integrate when error small (near target):

```
if (abs(error) < 0.010) {
    integral += error * dt;
}
```

Method 3: Back-Calculation Anti-Windup When output saturates, reduce integral:

```
if (output > MAX_OUTPUT) {
    integral -= (output - MAX_OUTPUT) / Ki;
    output = MAX_OUTPUT;
}
```

Method 4: Output Saturation Feedback (most common in servo drives) Stop integrating when output saturates.

Integral Gain Tuning

Start with Zero: - Tune P and D terms first - System should be stable but with steady-state error

Add Small Integral: - Start with $K_I = K_P / 10$ - Typical starting value: $K_I = 5-20$

Increase Slowly: - Increase by 20-50% each iteration - Watch for oscillation or excessive overshoot - Stop when steady-state error eliminated

Optimal K_I : - Just high enough to eliminate steady-state error - Not so high that it causes overshoot or slow settling - Typical: $K_I = (0.1-0.3) \times K_P$

Example: - $K_P = 75$, $K_D = 10$ - With P-D only: steady-state error = 0.0005" - Add $K_I = 5$: error = 0.0002" after 1 second - Add $K_I = 10$: error = 0.00005" after 0.5 seconds - Add $K_I = 20$: overshoot increases, oscillation - **Select $K_I = 10-15$**

Complete PID Formula

Continuous Time

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt}$$

Discrete Time (Position Form)

$$u[k] = K_P e[k] + K_I \sum_{i=0}^k e[i] \Delta t + K_D \frac{e[k] - e[k-1]}{\Delta t}$$

Discrete Time (Velocity Form)

More numerically stable for embedded systems:

$$\Delta u[k] = K_P (e[k] - e[k-1]) + K_I e[k] \Delta t + K_D (e[k] - 2e[k-1] + e[k-2])$$

$$u[k] = u[k-1] + \Delta u[k]$$

Advantage: Avoids summing large arrays (integral computed incrementally)

Practical Digital PID (LinuxCNC Style)

Bias Term: Some systems add output bias for friction compensation:

$$u[k] = K_P e[k] + K_I I[k] + K_D \frac{dy[k]}{dt} + u_{bias}$$

Deadband: Ignore very small errors (avoid dither):

```
if (abs(error) < DEADBAND) error = 0;
```

Typical deadband: 0.00005-0.0001" (1-2 encoder counts)

Units and Scaling

Consistent Units Critical: - Position: inches (or mm) - Velocity: inches/second (or mm/s) - Time: seconds

Example Unit Analysis: - $e = \text{inches}$ - $K_P = (\text{in/s}) / \text{in} = \text{s}^{-1}$ - $K_I = (\text{in/s}) / (\text{in} \cdot \text{s}) = \text{s}^{-2}$ - $K_D = (\text{in/s}) / (\text{in/s}) = \text{dimensionless}$

LinuxCNC Scaling: - Internal position units: “machine units” (counts, inches, mm - configurable) - Gains scaled by position loop period - Effective P gain = $K_P / (\text{loop frequency})$

PID Tuning Effects Summary

Increasing Proportional Gain (K_P)

Effects: - [check] Faster rise time - [check] Smaller steady-state error (but not zero) - x Increased overshoot - x Potential instability (oscillation)

Use When: Response too slow, large steady-state error

Increasing Integral Gain (K_I)

Effects: - [check] Eliminates steady-state error - [check] Compensates for constant disturbances - x Increased overshoot - x Slower settling time if too high - x Potential instability

Use When: Persistent steady-state error, gravity/friction compensation needed

Increasing Derivative Gain (K_D)

Effects: - [check] Reduced overshoot (more damping) - [check] Improved stability (allows higher K_P) - [check] Faster settling time - x Amplifies noise (if too high or unfiltered) - x Can cause “nervous” behavior

Use When: Excessive overshoot, oscillatory response

Complete PID Tuning Procedure (Manual Method)

Step 1: Prepare System

Mechanical: - All axes moving freely (no binding) - Couplings tight, no backlash - Encoders properly mounted and aligned - Wipers, guards removed (or loose)

Electrical: - Motor properly phased (for BLDC/AC servo) - Encoder direction matches motor direction - Drive enabled and responding - Emergency stop functional

Software: - Position loop running at 1-2 kHz - Soft limits disabled (for initial testing) - Following error limit set high (0.050-0.100”) - Encoder scale factor correct (test: jog 1”, measure actual distance)

Step 2: Set All Gains to Zero

Start with clean slate: - $K_P = 0$ - $K_I = 0$ - $K_D = 0$

Verify: Motor should not respond to position commands (no feedback).

Step 3: Tune Proportional Gain

3a. Initial P Gain: - Set $K_P = 10-20$ (conservative) - Command small move (0.100-0.500”) - Observe response (use Halscope, oscilloscope, or encoder readout)

3b. Increase P Gain: - If stable (no oscillation), increase by 50% - Repeat until one of: - Sustained oscillation appears -> reduce 30% and stop - Overshoot >20% -> note value, continue to Step 4 - Following error acceptable for application -> note value, continue

3c. Record P-Only Response: - Overshoot: _____% - Settling time: _____ ms - Steady-state error: _____ inches

Target: K_P value with 10-20% overshoot or just below oscillation onset.

Example: - $K_P = 100 \rightarrow$ 25% overshoot, ringing - $K_P = 75 \rightarrow$ 12% overshoot, damps quickly - **Select** $K_P = 75$ for derivative tuning

Step 4: Tune Derivative Gain

4a. Initial D Gain: - Set $K_D = K_P / 10$ (start) - Command same test move - Observe overshoot reduction

4b. Increase D Gain: - If overshoot still excessive, increase K_D by 25-50% - Repeat until overshoot <5-10% - Watch for “nervous” behavior (noise amplification)

4c. Optimal D Gain: - Overshoot: 2-8% (slight underdamping) - Smooth, well-damped response - No high-frequency jitter

Example: - $K_P = 75, K_D = 0 \rightarrow$ 12% overshoot - $K_P = 75, K_D = 5 \rightarrow$ 8% overshoot - $K_P = 75, K_D = 10 \rightarrow$ 3% overshoot - $K_P = 75, K_D = 15 \rightarrow$ 1% overshoot, getting jittery - **Select** $K_P = 75, K_D = 10$

Note: Can now increase K_P further for faster response (derivative adds damping).

4d. Retune P with D: - With $K_D = 10$, try $K_P = 100$ - If stable with <10% overshoot, even faster response achieved!

Step 5: Tune Integral Gain

5a. Verify Steady-State Error Exists: - Command move to position, hold - Wait 2-5 seconds - Measure final position error - Typical: 0.0002-0.002” error with P-D only

5b. Initial I Gain: - Set $K_I = K_P / 20$ (very conservative) - Command move, observe - Error should approach zero over several seconds

5c. Increase I Gain: - If error elimination too slow, increase K_I by 50% - If overshoot increases excessively, reduce K_I - Target: Error <0.0001” within 0.5-2 seconds

5d. Check for Overshoot: - Integral can cause overshoot on rapid moves - If overshoot becomes problematic, reduce K_I - Balance: Fast error correction vs. acceptable overshoot

Example: - $K_P = 100, K_D = 10, K_I = 0 \rightarrow$ steady-state error = 0.0005” - Add $K_I = 5 \rightarrow$ error = 0.0001” after 2 seconds, no extra overshoot - Add $K_I = 10 \rightarrow$ error = 0.00003” after 1 second, 5% overshoot - Add $K_I = 20 \rightarrow$ error = 0 after 0.5 seconds, 10% overshoot, ringing - **Select** $K_I = 8-10$

Step 6: Verify Performance

Test Moves: - Small moves (0.100”) - Medium moves (1.000”) - Large moves (full axis travel) - Rapid reversals (0.500” back and forth)

Check: - No oscillation at any position - Following error <0.001-0.002” during motion - Settling time acceptable - No “nervous” behavior

Load Testing: - Add workpiece weight (if applicable) - Cutting forces (make test cuts) - Verify stability under load

Step 7: Fine-Tuning and Optimization

Increase All Gains Proportionally: - If system very stable, can increase all gains 10-20% - Faster response while maintaining stability

Adjust for Different Operations: - Some systems use different gains for rapid vs. cutting - Higher gains for rapids (speed priority) - Lower gains for cutting (stability priority)

Document Final Values: | Gain | Value | Units | Notes | |——|——|——|——| | K_P | 100 | s^{-1} |
Proportional | | K_I | 10 | s^{-2} | Integral | | K_D | 10 | 1 | Derivative | | Deadband | 0.0001 | in | Error threshold
| | FF1 | 0.95 | 1 | Velocity feedforward (Section 19.5) | | Bias | 0.02 | in/s | Friction compensation |

PID Variants and Modifications

PI-D Control (Derivative on Measurement)

Instead of:

$$u = K_P e + K_I \int e \, dt + K_D \frac{de}{dt}$$

Use:

$$u = K_P e + K_I \int e \, dt - K_D \frac{dy}{dt}$$

Advantage: Eliminates derivative kick on setpoint changes.

I-PD Control (Proportional on Measurement)

$$u = K_I \int e \, dt + K_P(r - y) - K_D \frac{dy}{dt}$$

Some systems implement as:

$$u = K_I \int e \, dt - K_P y - K_D \frac{dy}{dt} + K_P r$$

Advantage: Further reduces setpoint change response (very smooth).

Parallel vs Series PID

Parallel (standard, described above):

$$u = K_P e + K_I \int e \, dt + K_D \frac{de}{dt}$$

Series (interacting):

$$u = K_c \left[e + \frac{1}{T_i} \int e \, dt + T_d \frac{de}{dt} \right]$$

where K_c = controller gain, T_i = integral time, T_d = derivative time

Conversion: - $K_P = K_c$ - $K_I = K_c/T_i$ - $K_D = K_c \cdot T_d$

Note: Most CNC systems use parallel form.

Setpoint Weighting

Some controllers allow weighting setpoint vs. measurement in P and D terms:

$$u = K_P(b \cdot r - y) + K_I \int e \, dt + K_D(c \cdot \frac{dr}{dt} - \frac{dy}{dt})$$

where b, c = weighting factors (0-1)

Effect: Reduces response magnitude to setpoint changes (smoother).

Typical: $b = 0.5, c = 0$ (no derivative on setpoint)

Common PID Tuning Problems

Problem 1: Oscillation (Instability)

Symptom: Sustained or growing oscillation around setpoint

Causes: - K_P too high - K_D too low (insufficient damping) - K_I too high (destabilizes)

Solution: - Reduce K_P by 30-50% - Increase K_D by 50-100% - Reduce K_I to zero, retune

Problem 2: Sluggish Response

Symptom: Slow rise time, long settling time

Causes: - K_P too low - K_D too high (overdamped) - Mechanical issues (binding, friction)

Solution: - Increase K_P by 50-100% - Reduce K_D by 30-50% - Check mechanical freedom

Problem 3: Persistent Steady-State Error

Symptom: Error remains after motion stops

Causes: - K_I too low or zero - Deadband too large - Friction or load exceeds motor torque

Solution: - Increase K_I - Check deadband setting - Verify motor capability

Problem 4: Excessive Overshoot

Symptom: Axis overshoots target significantly

Causes: - K_P too high - K_I too high (windup) - K_D too low

Solution: - Reduce K_P slightly - Reduce K_I (or add anti-windup) - Increase K_D

Problem 5: Noise Amplification (“Nervous” Axis)

Symptom: High-frequency jitter, audible whine from motor

Causes: - K_D too high - Encoder noise or resolution too coarse - No derivative filtering

Solution: - Reduce K_D by 30-50% - Add low-pass filter to derivative - Check encoder mounting (vibration)

Problem 6: Following Error During Motion

Symptom: Position lags command during constant velocity

Causes: - K_P too low - No velocity feedforward (Section 19.5) - Friction higher than expected

Solution: - Increase K_P - Add velocity feedforward (FF1) - Lubricate ways/screws

PID in CNC Context

Position Loop Only (Basic)

CNC Controller generates position commands at 1-2 kHz - PID loop calculates velocity command - Servo drive executes velocity (or torque) command

Common in: Hobby CNC, LinuxCNC, Mach4

Cascaded Loops (Advanced)

Outer Loop (position): CNC controller, 1-2 kHz **Inner Loop** (velocity or current): Servo drive, 8-16 kHz

Tuning: 1. Tune inner loop first (drive manufacturer pre-tunes) 2. Tune outer loop (user-accessible)

Common in: Industrial servo drives (Yaskawa, Delta, Panasonic)

Following Error Limit

Critical Safety Feature: Alarm if position error exceeds threshold

Typical Settings: - Warning: 0.010-0.020" (alert, no stop) - Fault: 0.050-0.100" (emergency stop, disable drives)

Purpose: Detect crashes, stalls, runaway motion

Example: - Axis encounters obstruction - Motor stalls, position lags - Following error grows: 0.005" -> 0.020" -> 0.050" - At 0.050": Controller issues FOLLOWING ERROR FAULT - Drives disable, motion stops

Summary

PID control provides the foundation for servo position control in CNC systems:

Three Terms: - **P:** Responds proportionally to error (speed, stiffness) - **I:** Eliminates steady-state error (compensates disturbances) - **D:** Adds damping (reduces overshoot, improves stability)

Tuning Process: 1. Tune P first (until oscillatory) 2. Add D to dampen (reduce overshoot) 3. Add I to eliminate steady-state error 4. Verify performance under all conditions

Key Insights: - Start with low gains, increase gradually - Test after each change - Balance speed vs. stability - Document final values

Next Steps: - Learn systematic tuning methods (Section 19.4) - Add feedforward for following error reduction (Section 19.5) - Implement trajectory planning (Sections 19.6-19.9)

Next: 19.4 PID Tuning Methods

19.4 PID Tuning Methods

Overview of Tuning Methods

PID tuning can be accomplished through several approaches, each with advantages and trade-offs:

Method	Complexity	Accuracy	Time Required	Equipment
Manual	Low	Moderate	30-60 min	Oscilloscope helpful
Ziegler-Nichols	Low	Moderate	15-30 min	None special
Relay Auto-Tune	Medium	Good	5-15 min	Software support
Software-Assisted	Medium	Excellent	10-30 min	Control software
Model-Based	High	Excellent	Variable	System ID tools

This Section Covers: 1. Manual tuning (already covered in 19.3) 2. Ziegler-Nichols method (classic, widely used) 3. Relay auto-tuning (automated) 4. Software-assisted tuning (LinuxCNC, Mach4) 5. Advanced model-based methods

Ziegler-Nichols Tuning Method

Background

Developed by John Ziegler and Nathaniel Nichols in 1942, this method provides a systematic procedure for finding PID gains based on system response characteristics.

Two Variants: 1. **Ultimate Gain Method** (closed-loop): Find gain at onset of oscillation 2. **Reaction Curve Method** (open-loop): Measure step response

Advantages: - Simple, well-established - No mathematical model required - Works for wide variety of systems

Disadvantages: - Aggressive tuning (often 25-50% overshoot) - Requires bringing system to edge of instability (risky) - May need refinement for CNC applications

Ultimate Gain Method (Closed-Loop)

Procedure:

Step 1: Set I and D to Zero - $K_I = 0$ - $K_D = 0$ - P-only control

Step 2: Increase K_P Until Sustained Oscillation - Start with low K_P (e.g., 10) - Gradually increase until system oscillates with constant amplitude - Record **ultimate gain** K_u and **oscillation period** T_u

Example: - Increase K_P : 10, 20, 40, 60, 80, 100... - At $K_P = 120$: Sustained oscillation appears - Measure period: $T_u = 0.15$ seconds - Ultimate gain: $K_u = 120$

Step 3: Calculate PID Gains

Ziegler-Nichols Formulas:

Controller Type	K_P	K_I	K_D
P	$0.5K_u$	0	0
PI	$0.45K_u$	$1.2K_P/T_u$	0
PID	$0.6K_u$	$2K_P/T_u$	$K_P T_u/8$

Example Calculation (PID): - $K_u = 120$, $T_u = 0.15$ s - $K_P = 0.6 \times 120 = 72$ - $K_I = 2 \times 72/0.15 = 960$ - $K_D = 72 \times 0.15/8 = 1.35$

Step 4: Test and Refine - Apply calculated gains - Test step response - Typical result: 10-25% overshoot, fast response - Reduce gains 20-30% if too aggressive for application

Safety Note: Bringing system to oscillation can be dangerous. Use: - Low mass or inertia for initial testing - Emergency stop readily accessible - Mechanical stops or soft limits active - Conservative gain increases (10-20% steps near K_u)

Reaction Curve Method (Open-Loop)

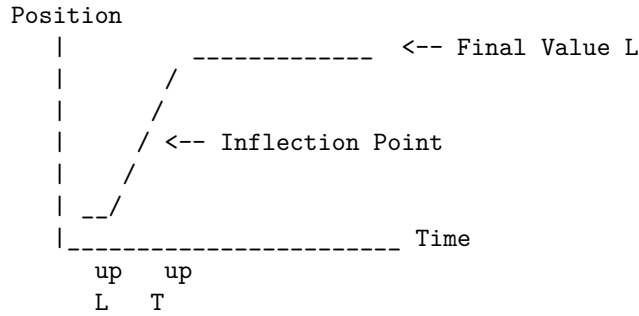
When to Use: System unstable or unsafe to oscillate under closed-loop P control.

Procedure:

Step 1: Open-Loop Step Response - Disable feedback (open loop) - Apply small step input to actuator (motor) - Record position vs. time

Step 2: Identify Response Parameters

Typical S-shaped response curve:



Measure: - L = Dead time (delay before response begins) - T = Time constant (time from inflection point to 63% of final value) - K = DC gain (final output / input step size)

Alternative: Tangent Method - Draw tangent line at steepest point - L = intersection with time axis - T = time from L to intersection with final value

Step 3: Calculate Gains

Ziegler-Nichols Reaction Curve Formulas:

Controller	K_P	K_I	K_D
P	$\frac{T}{L \cdot K}$	0	0
PI	$0.9 \frac{T}{L \cdot K}$	$\frac{K_P}{3.3L}$	0
PID	$1.2 \frac{T}{L \cdot K}$	$\frac{K_P}{2L}$	$0.5 K_P L$

Example: - Step input: 1.0 (normalized) - Dead time: $L = 0.02$ s - Time constant: $T = 0.08$ s - DC gain: $K = 0.8$

PID Gains: - $K_P = 1.2 \times 0.08 / (0.02 \times 0.8) = 6.0$ - $K_I = 6.0 / (2 \times 0.02) = 150$ - $K_D = 0.5 \times 6.0 \times 0.02 = 0.06$

Limitations: - Requires open-loop control (not always practical) - Assumes first-order + dead-time model (may not fit well) - Often too aggressive for CNC (needs detuning)

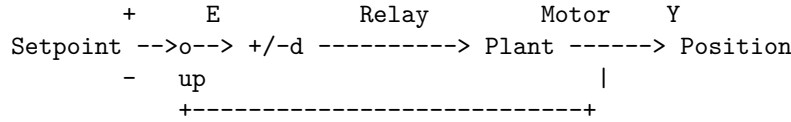
Relay Auto-Tuning

Principle

Relay Feedback Test: Replace controller with on-off relay, system naturally oscillates at critical frequency.

Advantages: - Automated (no manual gain adjustment) - Safer than Ziegler-Nichols ultimate gain method (limited relay output) - Fast (5-15 minutes typical) - Accurate identification of K_u and T_u

Process:



Relay outputs $+d$ if error positive, $-d$ if error negative.

Result: System oscillates with period T_u

Ultimate Gain Calculation:

$$K_u = \frac{4d}{\pi a}$$

where: - d = relay amplitude (magnitude of output) - a = oscillation amplitude (measured from position response)

Example: - Relay output: ± 1.0 (normalized) - Measured oscillation amplitude: $a = 0.0075$ inches - Period: $T_u = 0.12$ seconds

$$K_u = \frac{4 \times 1.0}{\pi \times 0.0075} = 170$$

Apply Ziegler-Nichols formulas with $K_u = 170$, $T_u = 0.12$ s.

Implementation

Algorithm: 1. Move to mid-position (allow oscillation both directions) 2. Apply relay feedback 3. Wait for sustained oscillation (3-5 cycles) 4. Measure amplitude and period 5. Calculate K_u , T_u 6. Compute PID gains using formulas 7. Switch to PID control with calculated gains

Refinements: - **Hysteresis:** Add deadband to relay (reduces chattering) - **Pre-load:** Add bias to relay output (compensate friction) - **Multiple relays:** Test at different amplitudes (check linearity)

Software Support: - Some industrial servo drives have built-in auto-tune (relay or similar method) - LinuxCNC: External scripts/HAL components - Mach4: Plugin support

Cohen-Coon Tuning Method

Alternative to Ziegler-Nichols: Better for processes with large dead time (L/T ratio > 0.3).

Based on: Open-loop reaction curve (same as Z-N reaction curve method)

Formulas (PID):

$$K_P = \frac{T}{L \cdot K} \left(1.35 + \frac{0.25L}{T} \right)$$

$$K_I = K_P \frac{30 + 3(L/T)}{9 + 20(L/T)} \frac{1}{L}$$

$$K_D = K_P \frac{4}{11 + 2(L/T)} L$$

When to Use: Systems with significant transport delay (e.g., temperature control, large pneumatic systems).

CNC Context: Rarely needed (CNC servo systems typically have small dead time).

Lambda Tuning (IMC Method)

Internal Model Control (IMC) or Lambda Tuning: Tune based on desired closed-loop time constant.

Philosophy: Specify desired response speed, calculate gains to achieve it.

Parameter: λ = desired closed-loop time constant (user-specified)

For First-Order + Dead-Time Model:

$$K_P = \frac{T}{K(\lambda + L)}$$

$$K_I = \frac{K_P}{T}$$

$$K_D = 0 \text{ (typically; or small value)}$$

Choosing λ : - **Small λ :** Fast response, aggressive (may overshoot or oscillate) - **Large λ :** Slow response, conservative (robust) - **Rule of thumb:** $\lambda = L$ to $\lambda = 3L$

Example: - Dead time: $L = 0.02$ s - Time constant: $T = 0.08$ s - DC gain: $K = 0.8$ - Choose: $\lambda = 0.03$ s (1.5 x dead time)

$$K_P = \frac{0.08}{0.8 \times (0.03 + 0.02)} = 2.0$$

$$K_I = \frac{2.0}{0.08} = 25$$

Advantages: - Intuitive parameter (λ = desired speed) - Generally more conservative than Ziegler-Nichols - Explicit robustness vs. performance trade-off

Disadvantages: - Requires system model (T, L, K) - May need iteration to find best λ

Software-Assisted Tuning

LinuxCNC Halscope Method

LinuxCNC provides excellent tools for servo tuning:

Tools: - **Halscope:** Real-time oscilloscope for HAL signals - **HAL:** Hardware Abstraction Layer (connects signals) - **PID Component:** Built-in PID loop

Procedure:

Step 1: Configure Halscope - Monitor signals: - Commanded position - Actual position (encoder feedback) - Following error - PID output (motor command) - Trigger on position command change

Step 2: Set Initial Gains - Use conservative values: - P = 50-100 - I = 0 - D = 0 - FF1 (velocity feedforward) = 0

Step 3: Tune Proportional Gain - Command small jog (0.1-0.5 inches) - Observe following error in Halscope - Increase P until response is fast with slight overshoot (5-10%)

Step 4: Add Velocity Feedforward (FF1) - Set $FF1 = 0.9-1.0$ (start) - Jog at constant velocity - Observe following error during constant-velocity portion - Adjust FF1 until following error near zero during motion - **Goal:** Following error < 0.001 " during 200 IPM rapid

Step 5: Add Derivative - Add $D = P/10$ (start) - Increase D until overshoot reduced to $< 5\%$ - Watch for noise amplification (jittery motion)

Step 6: Add Integral - Add $I = P/20$ (start) - Increase until steady-state error eliminated - Check for overshoot increase

Step 7: Iterate - Now that D and I are active, can increase P further (faster response) - Iterate between P, I, D adjustments - Test at various speeds and loads

Example LinuxCNC HAL Configuration:

```
# PID gains for X-axis
setp pid.x.Pgain 100
setp pid.x.Igain 10
setp pid.x.Dgain 8
setp pid.x.FF1 0.95
setp pid.x.deadband 0.0001
setp pid.x.maxoutput 10.0
```

Mach4 Tuning

Mach4 Motor Tuning:

Tools: - Built-in motor tuning wizard - Jogging controls - Position display

Procedure:

Step 1: Motor Configuration - Open Motor Configuration for axis - Set steps per unit (encoder resolution) - Set maximum velocity, acceleration

Step 2: Initial Gains - P Gain (Proportional): 100-200 - I Gain (Integral): 0-10 - D Gain (Derivative): 0

Step 3: P Gain Tuning - Jog axis back and forth - Increase P until motion is responsive - If oscillates, reduce P by 25-30%

Step 4: I Gain Tuning - Add small I gain (5-20) - Check for hunting (slow oscillation) - Reduce if unstable

Step 5: D Gain Tuning - Add D gain (10-50) - Improves damping - Watch for noise amplification

Step 6: Velocity Feedforward - Some drives support FF (check documentation) - Set $FF = 0.9-1.0$ - Reduces following error during motion

Step 7: Test - Run test programs (circles, squares, rapids) - Check following error display - Verify smooth motion at all speeds

Commercial Servo Drive Auto-Tune

Many Industrial Drives Include Auto-Tune:

Examples: - **Yaskawa Sigma-7:** Automatic gain tuning function - **Delta ASDA-A2:** Auto-tuning via drive parameters - **Panasonic MINAS A6:** One-touch tuning - **Kollmorgen AKD:** Auto-tune via software

Typical Auto-Tune Process: 1. Set motor parameters (inertia, rated specs) 2. Set load inertia ratio (or auto-detect) 3. Select stiffness level (1-100 or similar) 4. Run auto-tune routine 5. Drive performs identification (step, frequency sweep, or relay) 6. Drive calculates and sets gains automatically

User Input: - **Stiffness/Response Level:** 1 = soft (slow), 100 = stiff (fast) - **Inertia Ratio:** Load inertia / Motor inertia - Low inertia ratio (1-5): Higher gains possible - High inertia ratio (10-30): Lower gains required (stability)

Example (Yaskawa): - Parameter Pn102: Auto-tuning mode - 0 = Manual tuning - 1 = Auto-tuning (low response) - 2 = Auto-tuning (standard) - 3 = Auto-tuning (high response) - Execute: Set Pn102 = 2, cycle power or issue tune command - Drive runs auto-tune (30-60 seconds) - Gains automatically updated

Advantages: - Fast, automated - Pre-configured for motor model - Accounts for drive bandwidth, current loop tuning

Disadvantages: - Requires drive support (not all drives have auto-tune) - May not account for mechanical resonances - “Black box” (can’t see tuning logic)

Model-Based Tuning

System Identification

Goal: Create mathematical model of plant from measured data.

Process: 1. Apply known input signal (step, sine sweep, random) 2. Measure output response 3. Fit transfer function model to data 4. Design controller based on model

Tools: - MATLAB System Identification Toolbox - Python `scipy.signal` + optimization - Octave (open-source MATLAB alternative)

Example (Step Response): - Apply 1.0V step to motor - Record position vs. time - Fit second-order model:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} - \text{Optimize } \omega_n, \zeta \text{ to minimize fit error}$$

Result: $\omega_n = 50$ rad/s, $\zeta = 0.2$

Controller Design: - Specify desired closed-loop poles (e.g., $\zeta = 0.7$, $\omega_n = 40$ rad/s) - Calculate PID gains to achieve desired poles - Verify via simulation before implementing

Frequency Response Identification

Sine Sweep Method: - Apply sinusoidal input at varying frequencies (1-100 Hz typical) - Measure amplitude ratio and phase shift at each frequency - Plot Bode diagram from measured data - Fit transfer function model

Example: - Sweep 0.1-100 Hz (logarithmic spacing, 20 points) - At each frequency, measure gain and phase - Result: Bode plot of actual system - Identify resonances, bandwidth, phase lag

Controller Design: - Use loop-shaping techniques - Design controller to achieve target phase margin (45-60 deg) - Maximize bandwidth without exciting resonances

Advanced: Use optimization to find PID gains that maximize closed-loop bandwidth subject to phase margin constraint.

Optimal Control (LQR)

Linear Quadratic Regulator (LQR): State-space optimal control method.

State-Space Model:

$$\dot{x} = Ax + Bu$$

$$y = Cx$$

Cost Function (to minimize):

$$J = \int_0^{\infty} (x^T Q x + u^T R u) dt$$

where: - Q = state weighting matrix (penalize position, velocity error) - R = control weighting matrix (penalize large control effort)

LQR Solution:

$$u = -Kx$$

where K = optimal gain matrix (computed via Riccati equation)

Tuning: Adjust Q and R matrices to balance performance vs. control effort.

Advantages: - Provably optimal (for given Q , R) - Handles multi-input, multi-output naturally - Guaranteed stability

Disadvantages: - Requires accurate state-space model - Requires state estimation (Kalman filter if not all states measured) - Not standard in CNC controllers (research/advanced topic)

CNC Application: Mainly in research, advanced industrial systems, robotics.

Tuning for Specific Applications

High-Speed Machining

Goals: - Maximum acceleration - Minimal following error - Smooth contouring

Tuning Strategy: - Higher P gain (fast response) - Moderate D gain (damping without noise) - High velocity feedforward (FF1 approx 0.98-1.0) - Acceleration feedforward (FF2) if supported - Aggressive trajectory planning (Section 19.7)

Trade-offs: - May sacrifice stability margin for speed - More sensitive to mechanical resonances - Requires excellent mechanical construction

Precision Positioning

Goals: - Sub-micron accuracy - No overshoot - Minimal steady-state error

Tuning Strategy: - Moderate P gain (avoid overshoot) - High D gain (critical damping, $\zeta \approx 1.0$) - Moderate I gain (eliminate error without oscillation) - Lower velocity/acceleration limits - Use linear scales if possible (eliminate screw errors)

Trade-offs: - Slower than high-speed tuning - Longer settling times acceptable

Heavy Cutting (High Load Variation)

Goals: - Stable under varying loads - Compensate for cutting forces - Avoid chatter

Tuning Strategy: - Moderate P gain (avoid exciting chatter) - Higher I gain (compensate for load disturbances) - Moderate D gain - Velocity feedforward helps maintain speed under load - Consider notch filters for chatter frequencies

Trade-offs: - Less aggressive than high-speed tuning - Stability prioritized over raw speed

Troubleshooting Tuning Problems

Problem: Cannot Achieve Stable Tuning

Symptoms: Oscillation at any reasonable gain values

Possible Causes: 1. **Mechanical resonance:** Flexible structure, poor coupling 2. **Encoder mounting:** Vibration, loose mounting 3. **Wrong direction:** Positive feedback instead of negative 4. **Electrical noise:** Encoder signal corruption 5. **Insufficient loop rate:** Controller too slow (<500 Hz)

Solutions: - Tap test: Identify resonances (hammer tap, measure ring-down) - Add mechanical stiffness (bracing, better couplings) - Check encoder direction vs. motor direction (must match!) - Shielded encoder cables, proper grounding - Increase control loop rate if possible

Problem: Excessive Following Error

Symptoms: Position lags command by 0.005-0.050" during motion

Possible Causes: 1. K_P too low 2. No velocity feedforward 3. Excessive friction 4. Motor undersized

Solutions: - Increase P gain (if stable) - Add velocity feedforward ($FF1 = 0.9-1.0$) - Lubricate ways, check for binding - Verify motor torque adequate for acceleration required

Problem: Steady-State Error Persists

Symptoms: 0.001-0.005" error after motion stops

Possible Causes: 1. K_I too low or zero 2. Deadband too large 3. Stiction (static friction) exceeds motor torque

Solutions: - Increase integral gain - Reduce deadband (0.00005-0.0001" typical) - Reduce friction, add bias term for gravity compensation

Problem: Noisy/Jittery Motion

Symptoms: High-frequency vibration, audible whine

Possible Causes: 1. K_D too high 2. No derivative filtering 3. Encoder resolution too coarse 4. Electrical noise on encoder

Solutions: - Reduce derivative gain 30-50% - Add or increase derivative filter time constant - Upgrade encoder (higher resolution) - Check cabling, grounding, shielding

Summary

Multiple PID tuning methods available, each suited to different situations:

Ziegler-Nichols: Classic, simple, but aggressive (requires refinement)

Relay Auto-Tune: Automated, safe, fast (needs software support)

Software-Assisted: Best for CNC (LinuxCNC Halscope, Mach4 tools)

Model-Based: Most accurate, requires system identification

General Recommendations: 1. Start conservative (low gains) 2. Tune P first, then D, then I 3. Add feedforward for following error reduction 4. Test under realistic conditions 5. Document final values

Next Steps: - Implement advanced control techniques (Section 19.5) - Design optimal trajectories (Sections 19.6-19.9) - Configure LinuxCNC or Mach4 (Sections 19.10-19.11)

Next: 19.5 Advanced Control Techniques

19.5 Advanced Control Techniques

Beyond Basic PID

Basic PID control provides excellent performance for many CNC applications, but advanced techniques can further improve:

- **Following error reduction:** Feedforward control
- **Resonance suppression:** Notch and low-pass filters
- **Load compensation:** Adaptive control
- **Multi-axis coordination:** Cross-coupling control
- **Precision enhancement:** Dual-loop control, backlash compensation

This section covers practical advanced techniques applicable to CNC servo systems.

Feedforward Control

The Feedforward Concept

Fundamental Limitation of Feedback: PID control is inherently **reactive** – it responds to errors after they occur.

Feedforward Solution: Add control terms based on **commanded motion** (not error).

Analogy: Driving a car - **Feedback only:** Wait until speed drops below target, then press gas (slow response) - **Feedforward:** Anticipate hill ahead, press gas before speed drops (proactive)

CNC Application: - **Feedback (PID):** Corrects position errors - **Feedforward:** Anticipates required motor torque for commanded motion

Result: Dramatically reduced following error during motion.

Velocity Feedforward (FF1)

Principle: During constant velocity motion, motor must produce torque to overcome friction and viscous damping.

Without FF1: - Commanded velocity: 100 IPM - Actual position lags by 0.005" (following error) - PID works hard to correct this persistent error

With FF1: - Add command proportional to commanded velocity - Motor immediately receives correct command for steady-state velocity - Following error reduced to 0.0005" (10x improvement!)

Formula:

$$u_{FF1} = FF1 \times \dot{r}(t)$$

where: - FF1 = velocity feedforward gain - $\dot{r}(t)$ = commanded velocity

Total Control:

$$u_{total} = K_P e + K_I \int e dt + K_D \frac{de}{dt} + FF1 \cdot \dot{r}$$

Tuning FF1:

Step 1: Tune PID first (P, D, I)

Step 2: Command constant-velocity motion (e.g., 200 IPM jog)

Step 3: Observe following error during constant velocity portion

Step 4: Increase FF1 from 0 to 1.0 (typically) - FF1 = 0: No feedforward (baseline following error) - FF1 = 0.5: Following error reduced by ~50% - FF1 = 1.0: Following error minimal (may need fine-tuning) - FF1 > 1.0: Overshoot, position leads command (too much)

Optimal FF1: Following error during constant velocity $< 0.001''$ (± 1 encoder count)

Typical Values: FF1 = 0.9-1.0 for well-tuned systems

Example: - Without FF1: Following error = $0.006''$ at 300 IPM - With FF1 = 0.95: Following error = $0.0005''$ at 300 IPM - **12x improvement**

LinuxCNC Configuration:

```
setp pid.x.FF1 0.95
```

Acceleration Feedforward (FF2)

Principle: During acceleration, motor must produce torque proportional to acceleration (Newton's 2nd law: $F = ma$).

Without FF2: - Commanded acceleration: 200 in/s^2 - Position lags during acceleration phase - Following error peaks during acceleration (transient error)

With FF2: - Add command proportional to commanded acceleration - Motor receives correct torque for acceleration instantly - Following error during acceleration reduced

Formula:

$$u_{FF2} = \text{FF2} \times \ddot{r}(t)$$

where: - FF2 = acceleration feedforward gain - $\ddot{r}(t)$ = commanded acceleration

Total Control (with FF1 and FF2):

$$u_{total} = K_P e + K_I \int e dt + K_D \frac{de}{dt} + \text{FF1} \cdot \dot{r} + \text{FF2} \cdot \ddot{r}$$

Tuning FF2:

Step 1: Tune PID and FF1 first

Step 2: Command trapezoidal move (acceleration $\rightarrow$ constant velocity $\rightarrow$ deceleration)

Step 3: Observe following error during acceleration and deceleration phases

Step 4: Increase FF2 from 0 - Start with FF2 = 0.0001-0.001 (small values) - Increase until transient following error minimized - Too high: Overshoot during acceleration

Optimal FF2: Following error spike during acceleration < 0.001 - $0.002''$

Typical Values: FF2 = 0.0001-0.005 (depends on inertia and units)

Example: - $1''$ rapid move, 200 in/s^2 acceleration - Without FF2: Peak following error = $0.004''$ during acceleration - With FF2 = 0.002: Peak following error = $0.001''$ during acceleration - **4x improvement in transient response**

LinuxCNC Configuration:

```
setp pid.x.FF1 0.95
```

```
setp pid.x.FF2 0.002
```

Feedforward Gain Calculation (Theoretical)

Velocity Feedforward (first-order plant):

For plant $G(s) = \frac{K}{\tau s + 1}$:

$$\text{FF1} = \frac{1}{K}$$

Acceleration Feedforward (second-order plant):

For plant $G(s) = \frac{1}{ms^2 + bs + k}$ (mass-spring-damper):

$$\text{FF2} = m$$

CNC Context: - m = moving mass + reflected load inertia - b = viscous damping - k = spring stiffness (if flexible coupling)

Example: - Axis moving mass: 50 kg - Ballscrew pitch: 5 mm/rev - Motor inertia (reflected): 0.5 kg equivalent - Total effective mass: 50.5 kg

$$\text{FF2} = 50.5 \text{ kg} = 50.5 \text{ N}/(\text{m/s}^2)$$

Convert to system units (if needed).

Practical Note: Calculated values are starting points; empirical tuning gives best results.

Bias (Constant Offset)

Purpose: Compensate for constant disturbances (gravity on vertical axis, friction).

Formula:

$$u_{bias} = \text{constant}$$

Total Control:

$$u_{total} = \text{PID} + \text{FF1} \cdot \dot{r} + \text{FF2} \cdot \ddot{r} + u_{bias}$$

Example: Vertical Z-axis with 100 lb spindle head - Gravity force: 100 lb downward (always) - Without bias: Integral term accumulates to compensate (slow, windup risk) - With bias: Constant upward force = 100 lb (instantaneous compensation)

Tuning: - Move axis to mid-position, hold - Measure steady-state PID output - Set bias = measured output - Verify axis holds position with minimal integral accumulation

LinuxCNC Configuration:

```
setp pid.z.bias 2.5 # Units: velocity command (in/s typically)
```

Input Shaping

Concept

Problem: Step commands in position excite mechanical resonances (ringing).

Solution: Shape input command to avoid exciting resonances.

Method: Convolve step input with filter designed to cancel resonance.

Zero-Vibration (ZV) Shaper: - Split step command into two smaller steps - Timing and amplitude chosen to cancel resonance

Example: - System with resonance at 50 Hz (period = 0.02 s) - Instead of single step at $t = 0$: - Apply 0.5 step at $t = 0$ - Apply 0.5 step at $t = 0.01$ s (half period later) - Second pulse arrives when first pulse oscillation is at peak -> cancels

Implementation: - Some CNC controllers have built-in input shaping - LinuxCNC: External HAL component or trajectory planning - Mach4: Plugin support (limited)

Trade-off: Slightly slower response (delay = half resonance period), but eliminates ringing.

Application: Lightweight gantries, long unsupported axes (3D printers, pick-and-place)

Filtering Techniques

Low-Pass Filters

Purpose: Attenuate high-frequency noise and commands.

First-Order Low-Pass:

$$H(s) = \frac{1}{\tau s + 1}$$

Cutoff Frequency: $f_c = \frac{1}{2\pi\tau}$

Effect: - Frequencies below f_c : Pass through (minimal attenuation) - Frequencies above f_c : Attenuated (-20 dB/decade)

Application: - Filter derivative term (reduce noise amplification) - Filter command input (smooth jerky commands) - Filter encoder signal (reduce quantization noise)

Example: - Encoder quantization: +/-0.00005" jitter - Without filter: Derivative term amplifies noise - With 100 Hz low-pass on derivative: Noise attenuated, derivative still effective

LinuxCNC:

```
loadrt lowpass names=lowpass.d-term
setp lowpass.d-term.gain 1.0
setp lowpass.d-term.time-constant 0.002 # 2 ms = ~80 Hz cutoff
```

Notch Filters

Purpose: Eliminate specific frequency (resonance) from control loop.

Transfer Function:

$$H(s) = \frac{s^2 + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

where: - ω_n = notch center frequency (rad/s) - ζ = damping ratio (notch width)

Effect: - At ω_n : Gain approx 0 (complete rejection) - Away from ω_n : Gain approx 1 (passes through)

Frequency Response: - Narrow notch (high Q): $\zeta = 0.1$ (rejects +/-5 Hz around center) - Wide notch (low Q): $\zeta = 0.5$ (rejects +/-20 Hz around center)

Identification of Resonance Frequency:

Method 1: Tap Test - Tap axis with hammer (or similar) - Measure ring-down with accelerometer or encoder - FFT of response shows resonance peak

Method 2: Frequency Sweep - Apply sinusoidal command at varying frequencies - Measure response amplitude - Peak in response = resonance frequency

Method 3: Increase Gains Until Oscillation - Gradually increase P and D gains - Note frequency of oscillation when system becomes unstable - Resonance frequency approx oscillation frequency

Example: - Tap test shows resonance at 247 Hz - Design notch filter: $f_n = 247$ Hz, $\zeta = 0.15$ (narrow notch) - Implement in control loop - **Result:** Can increase PID gains 50-100% without exciting resonance

LinuxCNC:

```
# Notch filter for 247 Hz resonance
loadrt notch names=notch.x-axis
setp notch.x-axis.freq 247
setp notch.x-axis.q 6.67 # Q = 1/(2*zeta), zeta=0.075
```

Application: - Flexible gantries (100-300 Hz typical) - Long unsupported screws (50-150 Hz) - Spindle mounted on Z-axis (200-500 Hz)

Trade-off: Notch filter adds phase lag (reduces phase margin slightly). Use narrowest notch possible.

Bi-Quad Filter

Bi-Quadratic Filter: General second-order filter (can implement low-pass, high-pass, band-pass, notch).

Transfer Function:

$$H(s) = \frac{b_0 s^2 + b_1 s + b_2}{s^2 + a_1 s + a_2}$$

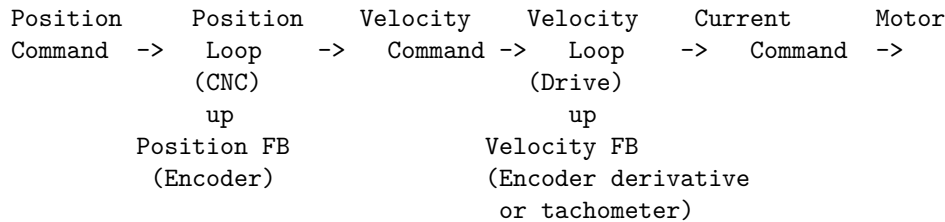
Advantage: Flexible, can implement multiple filter types with coefficient changes.

CNC Use: Less common than simple notch/low-pass (more complex to tune).

Dual-Loop Control

Position and Velocity Loops

Cascaded Control: Outer position loop commands inner velocity loop.



Division of Labor: - **Outer loop** (position): Slow (1-2 kHz), CNC controller - **Inner loop** (velocity): Fast (8-16 kHz), servo drive

Advantages: - Velocity loop bandwidth higher than position loop (faster disturbance rejection) - Velocity loop stabilizes motor (prevents runaway) - Easier to tune (tune inner loop first, then outer)

Tuning: 1. **Velocity loop** (drive): Manufacturer often pre-tunes 2. **Position loop** (CNC): User tunes P, I, D as usual

Example: - Velocity loop bandwidth: 500 Hz (drive internal) - Position loop bandwidth: 50 Hz (CNC controller) - **10:1 separation** (rule of thumb: inner loop 5-10x faster than outer)

Industrial Servo Drives: Almost always implement velocity loop internally (user tunes position loop only).

State-Space Control

Overview

State-Space Representation: Modern control theory, models system as first-order differential equations.

State Vector: $x = [position, velocity, ...]^T$

State Equations:

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

where: - A = state matrix (system dynamics) - B = input matrix - C = output matrix - D = feedthrough matrix

Example (mass-spring-damper):

$$\ddot{y} + 2\zeta\omega_n\dot{y} + \omega_n^2 y = u$$

State-space form ($x_1 = y$, $x_2 = \dot{y}$):

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$

Controller: $u = -Kx$ (state feedback)

where $K = [K_1, K_2]$ = gain vector

Equivalent to: - K_1 approx proportional gain (position error) - K_2 approx derivative gain (velocity error)

LQR (Linear Quadratic Regulator)

Optimal Control: Design K to minimize cost function:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

Tuning: Adjust Q (state weighting) and R (control effort weighting)

Solution: Solve Algebraic Riccati Equation (ARE) for K .

Advantages: - Provably optimal (for given Q, R) - Systematic design - Guaranteed stability

CNC Application: Research topic, not common in commercial controllers (LinuxCNC, Mach4 use PID).

When Useful: - Multi-axis coordinated control (gantry synchronization) - Complex system dynamics (flexible structures) - Academic/research CNC projects

Adaptive Control

Concept

Fixed Gains: Traditional PID has constant K_P , K_I , K_D .

Adaptive Gains: Controller adjusts gains automatically based on operating conditions.

Why Adaptive: - System dynamics change (load inertia varies, temperature affects friction) - Fixed gains optimized for one condition may be suboptimal for others - Adaptive control maintains performance across conditions

Gain Scheduling

Method: Pre-compute gains for multiple operating points, interpolate.

Example (Lathe): - Light workpiece (low inertia): $K_P = 200$ - Heavy workpiece (high inertia): $K_P = 100$ - Controller measures load inertia (from acceleration response) - Automatically adjusts K_P based on measurement

Implementation: - Create lookup table: Inertia vs. Gains - Measure or estimate inertia (online or offline) - Interpolate gains from table

Application: - Variable payload robots - Machine tools with different workpiece masses - Systems with configuration changes (e.g., gantry position affects dynamics)

Model Reference Adaptive Control (MRAC)

Concept: Define reference model (desired behavior), adapt gains to match.

Reference Model: Ideal system (e.g., critically damped second-order)

Adaptation Law: Continuously adjust controller gains to minimize tracking error vs. reference model.

Advantages: - Handles unknown plant parameters - Robust to parameter changes

Disadvantages: - Complex implementation - Requires careful stability analysis - Not common in CNC (research topic)

Cross-Coupling Control

Gantry Synchronization Problem

Dual-Motor Gantry (e.g., plasma table with two Y-axis motors):

Problem: Motors may not track perfectly, causing gantry to rack (skew).

Traditional Approach: Tune each motor independently.

Limitation: If one motor lags, gantry skews, causes binding and inaccuracy.

Cross-Coupling Solution

Add Cross-Coupling Term: Each motor's controller receives feedback from both motors.

Example: - Motor 1 position: y_1 - Motor 2 position: y_2 - **Cross-coupling error:** $e_{cc} = y_1 - y_2$

Motor 1 command:

$$u_1 = \text{PID}_1(r - y_1) - K_{cc} \cdot e_{cc}$$

Motor 2 command:

$$u_2 = \text{PID}_2(r - y_2) + K_{cc} \cdot e_{cc}$$

Effect: - If Motor 1 leads: $e_{cc} > 0$ -> reduce u_1 , increase u_2 (synchronize) - If Motor 2 leads: $e_{cc} < 0$ -> increase u_1 , reduce u_2

Result: Motors stay synchronized (gantry remains square).

Tuning K_{cc} : - Start low ($K_{cc} = 10\text{-}20\%$ of K_P) - Increase until synchronization error $< 0.001''$ - Too high: Can cause instability (fighting between axes)

LinuxCNC: HAL component `gantrykins` or custom HAL logic.

Application: - Dual-motor gantries (plasma, router, laser) - Coordinated multi-axis (parallel robots)

Backlash Compensation

The Backlash Problem

Backlash: Play/clearance in mechanical transmission (nut-screw gap, gear lash).

Effect: - Hysteresis in motion - Lost motion during direction reversal - Position error during bidirectional moves

Example: - Ballscrew has $0.002''$ backlash - Move +X, then -X - Actual position lags by $0.002''$ during reversal

Software Compensation

Method: Add backlash correction term when direction reverses.

Algorithm: 1. Detect direction change (velocity sign change) 2. Add offset = backlash amount 3. Gradually remove offset over short time/distance

LinuxCNC:

```
setp axis.x.backlash 0.002 # inches
```

Controller automatically compensates during direction reversals.

Limitations: - Compensation perfect only for slow moves - Fast reversals: Compensation incomplete (inertia, dynamics) - Better solution: **Eliminate backlash mechanically** (anti-backlash nut, preload)

When to Use: - Legacy machines with worn screws - Temporary fix until mechanical repair - Non-critical applications (not precision machining)

Summary

Advanced control techniques extend PID performance:

Feedforward (FF1, FF2): - Reduce following error 5-10x - Essential for high-speed machining - Easy to implement, huge benefit

Notch Filters: - Eliminate specific resonances - Allow higher gains (faster response) - Requires resonance identification (tap test, sweep)

Dual-Loop Control: - Fast inner loop (velocity), slow outer loop (position) - Standard in industrial servo drives - Easier tuning, better performance

Adaptive/Cross-Coupling: - Handle varying dynamics - Synchronize multi-motor axes - More complex, specialized applications

Practical Priority: 1. **PID tuning** (foundation) 2. **Feedforward** (FF1, FF2) (biggest bang for buck) 3. **Notch filters** (if resonances present) 4. **Advanced techniques** (as needed for specific applications)

Next Steps: - Apply advanced control to trajectory planning (Section 19.6) - Design optimal motion profiles (Section 19.7) - Implement in LinuxCNC/Mach4 (Sections 19.10-19.11)

Next: 19.6 Trajectory Planning Fundamentals

19.6 Trajectory Planning Fundamentals

What is Trajectory Planning?

Trajectory Planning: The process of generating time-based position, velocity, and acceleration commands that move a machine from one location to another while respecting kinematic and dynamic constraints.

Distinction: - **Path:** Geometric route through space (e.g., line from A to B, arc, spline) - **Trajectory:** Path + timing (position as function of time: $x(t)$, $y(t)$, $z(t)$)

Example: - **Path:** “Move in straight line from (0,0) to (10,10)” - **Trajectory:** “At $t=0s$, position=(0,0); at $t=0.5s$, position=(5,5); at $t=1.0s$, position=(10,10)”

Why Trajectory Planning Matters: - Respects machine limits (max velocity, acceleration, jerk) - Minimizes cycle time (fastest possible while staying within limits) - Ensures smooth motion (avoids jerks that excite vibration) - Coordinates multiple axes (maintain tool path accuracy)

Point-to-Point vs. Continuous Path Motion

Point-to-Point (PTP) Motion

Characteristics: - Move from start point to end point - Path between points doesn't matter - Each axis can move independently at maximum rate - Used for positioning, tool changes, rapids

Example: G0 rapid move

G0 X10 Y5 Z2

Behavior: - X, Y, Z axes accelerate independently - Fastest axis arrives first, waits for others - Actual path is NOT a straight line (unless carefully coordinated)

Typical Applications: - Rapid positioning between cuts - Tool changes (moving to/from tool changer) - Probing moves (to/from probe point)

Advantage: Fastest possible move (each axis at max speed)

Disadvantage: Cannot control actual path taken

Continuous Path (CP) Motion

Characteristics: - Path is precisely defined (line, arc, spline) - All axes coordinated to follow path exactly - Velocity along path may vary - Used for cutting, contouring

Example: G1 linear interpolation

G1 X10 Y5 F100

Behavior: - Controller calculates X and Y motion to produce straight line - Velocity along path = 100 IPM (feedrate) - X and Y velocities continuously adjusted to maintain path

Typical Applications: - Linear cuts (milling, turning) - Circular interpolation (arcs, holes) - Complex contours (splines, NURBS)

Advantage: Precise path control (geometric accuracy)

Disadvantage: Slower than PTP (axes must coordinate)

Kinematic Constraints

Velocity Limits

Maximum Velocity: Highest speed each axis can sustain.

Limits Set By: 1. **Motor speed:** Maximum RPM 2. **Mechanical:** Screw pitch, belt ratio 3. **Control:** Encoder resolution, update rate 4. **Thermal:** Continuous motor current limit

Example: - Motor max speed: 3000 RPM - Ballscrew pitch: 5 mm/rev (0.2 in/rev) - Max linear velocity: $3000 \times 0.2 = 600$ in/min

Per-Axis Limits: Each axis may have different max velocity. - X-axis: 600 IPM - Y-axis: 500 IPM (longer, heavier) - Z-axis: 300 IPM (vertical, gravity load)

Coordinated Motion: Must slow down to respect all axes.

Example: Diagonal move from (0,0) to (10,10) at F600 - Equal X and Y motion - If X max = 600 IPM, Y max = 500 IPM: - Cannot achieve F600 (would require Y = 424 IPM, OK) - **Achievable feedrate = 600 IPM** (within limits)

Example 2: Diagonal (0,0) to (10,5) - X motion = 10", Y motion = 5" - Path length = $\sqrt{10^2 + 5^2} = 11.18"$ - Desired F = 600 IPM - Required X velocity: $(10/11.18) \times 600 = 537$ IPM (OK) - Required Y velocity: $(5/11.18) \times 600 = 268$ IPM (OK) - **Achievable feedrate = 600 IPM**

Example 3: Diagonal (0,0) to (10,10) at F800 - Path length = $\sqrt{10^2 + 10^2} = 14.14"$ - Required X velocity: $(10/14.14) \times 800 = 566$ IPM (OK, < 600) - Required Y velocity: $(10/14.14) \times 800 = 566$ IPM (exceeds 500 max!) - **Must reduce feedrate:** $F = 500 \times 14.14/10 = 707$ IPM - Recalculate: X = Y = 500 IPM (at limit)

Acceleration Limits

Maximum Acceleration: Rate of velocity change each axis can achieve.

Limits Set By: 1. **Motor torque:** Peak and continuous ratings 2. **Inertia:** Moving mass + load 3. **Mechanical:** Frame rigidity, bearing friction 4. **Control:** Following error limits

Example: - Motor peak torque: 3.0 Nm - *Moving inertia:* 0.01 kgm² - Max angular acceleration: $3.0 / 0.01 = 300$ rad/s² - Ballscrew pitch: 5 mm/rev = 0.0318 m/rev - Max linear acceleration: $300 \times 0.0318 = 9.54$ m/s² approx 22,500 in/min² approx 375 in/s²

Typical CNC Accelerations: - Hobby/DIY: 50-100 in/s² - Industrial mill: 150-300 in/s² - High-speed router: 300-500 in/s² - Pick-and-place: 1000-3000 in/s²

Coordinated Acceleration: Must respect all axes during multi-axis moves.

Acceleration Distance (from rest to max velocity):

$$d = \frac{v_{max}^2}{2a_{max}}$$

Example: - Max velocity: 600 IPM = 10 in/s - Max acceleration: 200 in/s² - Acceleration distance: $10^2 / (2 \times 200) = 0.25$ inches

Implication: For moves < 0.5", never reaches max velocity (entirely accel + decel).

Jerk Limits

Jerk: Rate of change of acceleration.

$$j = \frac{da}{dt}$$

Why Limit Jerk: 1. **Smooth motion:** Sudden acceleration changes excite vibration 2. **Mechanical stress:** Reduces shock loads on bearings, frame 3. **Following error:** Gradual acceleration changes easier for servo to track 4. **Surface finish:** Smooth motion -> better finish

Unlimited Jerk: Trapezoidal velocity profile (instant acceleration change) - Fast cycle time - Harsh motion (vibration, ringing)

Limited Jerk: S-curve velocity profile (smooth acceleration change) - Slightly slower cycle time - Smooth motion (better surface finish)

Typical Jerk Limits: - Trapezoidal (no jerk limit): infinity - Moderate jerk limit: 5,000-10,000 in/s³ - Aggressive jerk limit: 20,000-50,000 in/s³ - High-speed machining: 100,000+ in/s³ (but still limited)

Trade-off: Lower jerk limit = smoother motion but longer cycle time.

Real-Time vs. Pre-Computed Trajectories

Real-Time Trajectory Generation

Method: Controller generates trajectory on-the-fly, synchronized with motion execution.

Characteristics: - Trajectory computed in real-time (every servo cycle, e.g., 1 kHz) - Can respond to external events (feedhold, feed override) - Requires fast processor, real-time OS

Example: LinuxCNC - Real-time trajectory planner - Updates trajectory every 1 ms (1 kHz servo thread) - Computes position, velocity, acceleration for each axis - Sends commands to PID loops

Advantages: - Dynamic response (feedhold stops immediately) - Feed rate override (real-time adjustment) - Adaptive feed (slow down for heavy cuts)

Disadvantages: - Requires real-time computing (not all systems support) - Limited look-ahead (computational constraints)

Pre-Computed Trajectories

Method: Entire toolpath trajectory computed before motion begins.

Characteristics: - Trajectory stored in memory (position vs. time table) - Playback during execution (no real-time computation) - Can optimize globally (entire path visible)

Example: CAM post-processor - Generates optimized trajectory (considers entire part) - Outputs G-code + velocity profile - Controller plays back pre-computed trajectory

Advantages: - Optimal trajectory (global optimization possible) - Simpler controller (playback only) - Deterministic (repeatable timing)

Disadvantages: - Cannot respond dynamically (feedhold = stop playback, then resume) - Large memory for complex parts - Less flexible

Hybrid Approach (Common in Modern CNC)

Method: Pre-compute short segments, update in real-time.

Example: - Read-ahead buffer: 50-200 G-code blocks - Compute optimal trajectory for buffered blocks - Update trajectory as new blocks arrive - Retain real-time adjustability (feedhold, override)

Best of Both: Optimization + dynamic response.

Interpolation

Linear Interpolation

G1 Command: Move in straight line at specified feedrate.

G1 X10 Y5 Z2 F120

Trajectory Generation: 1. Calculate path length: $L = \sqrt{(X_1 - X_0)^2 + (Y_1 - Y_0)^2 + (Z_1 - Z_0)^2}$ 2. Calculate unit direction vector: $\hat{u} = [(X_1 - X_0)/L, (Y_1 - Y_0)/L, (Z_1 - Z_0)/L]$ 3. Calculate time duration: $T = L/F$ (feedrate) 4. Generate trajectory: $\vec{r}(t) = \vec{r}_0 + \hat{u} \cdot v(t) \cdot t$

where $v(t)$ = velocity profile (constant or ramped with accel/decel)

Example: - Start: (0, 0, 0) - End: (10, 5, 0) - Feedrate: 120 IPM = 2 in/s - Path length: $\sqrt{100+25}$ = 11.18 inches - Duration: 11.18 / 2 = 5.59 seconds - X velocity: (10/11.18) x 2 = 1.789 in/s - Y velocity: (5/11.18) x 2 = 0.894 in/s

At each servo update (e.g., 1 ms): - $t = 0.000$ s: Position = (0.000, 0.000, 0.000) - $t = 0.001$ s: Position = (0.00179, 0.00089, 0.000) - $t = 0.002$ s: Position = (0.00358, 0.00179, 0.000) - ... - $t = 5.590$ s: Position = (10.000, 5.000, 0.000)

Circular Interpolation

G2/G3 Commands: Move in circular arc.

G2 X10 Y10 I5 J0 F100

Parameters: - (X, Y): Arc endpoint - (I, J): Center offset from start point - F: Feedrate

Trajectory Generation: 1. Calculate center: $(X_c, Y_c) = (X_0 + I, Y_0 + J)$ 2. Calculate radius: $R = \sqrt{I^2 + J^2}$ 3. Calculate start angle: $\theta_0 = \text{atan2}(Y_0 - Y_c, X_0 - X_c)$ 4. Calculate end angle: $\theta_1 = \text{atan2}(Y_1 - Y_c, X_1 - X_c)$ 5. Calculate arc length: $L = R \cdot |\theta_1 - \theta_0|$ 6. Generate trajectory: - $\theta(t) = \theta_0 + (\theta_1 - \theta_0) \cdot (s(t)/L)$ - $X(t) = X_c + R \cos(\theta(t))$ - $Y(t) = Y_c + R \sin(\theta(t))$

where $s(t)$ = arc distance traveled (from velocity profile)

Challenge: Velocity along arc creates centripetal acceleration.

Centripetal Acceleration:

$$a_c = \frac{v^2}{R}$$

Example: - Radius: 2 inches - Feedrate: 200 IPM = 3.33 in/s - Centripetal acceleration: $3.33^2 / 2 = 5.56$ in/s²

If max acceleration = 200 in/s², this is well within limits.

Small Radius, High Speed: - Radius: 0.1 inches - Feedrate: 200 IPM = 3.33 in/s - Centripetal acceleration: $3.33^2 / 0.1 = 111$ in/s² (still OK)

Very Small Radius: - Radius: 0.02 inches - Feedrate: 200 IPM = 3.33 in/s - Centripetal acceleration: $3.33^2 / 0.02 = 556$ in/s² (exceeds 200 limit!)

Controller must reduce feedrate to respect acceleration limits.

Allowable velocity (for given radius and max acceleration):

$$v_{max} = \sqrt{a_{max} \cdot R}$$

Example: $v_{max} = \sqrt{200 \times 0.02} = 2.0$ in/s = 120 IPM

Automatic Feedrate Reduction: Modern controllers slow down automatically for tight arcs.

Helical Interpolation

Helical Move: Circular arc in XY plane + linear Z motion.

G2 X10 Y10 Z-1 I5 J0 F80

Trajectory: - XY: Circular arc (as above) - Z: Linear interpolation (simultaneous) - Feedrate: Along 3D helical path

Application: - Thread milling - Helical hole entry (ramping into hole) - Spring-like toolpaths

Spline Interpolation

Spline: Smooth curve through multiple points.

Types: - **Cubic spline:** Piecewise cubic polynomials (continuous to 2nd derivative) - **B-spline:** Basis spline (localized control points) - **NURBS:** Non-Uniform Rational B-Splines (industry standard in CAD/CAM)

G-Code Support: - **G5:** Cubic spline (limited support) - **G5.1:** Quadratic B-spline (LinuxCNC) - **G5.2/G5.3:** NURBS (limited controllers)

Example (LinuxCNC G5.1):

```
G5.1 X5 Y2
G5.1 X8 Y6
G5.1 X10 Y4
```

Advantage: Smooth curves (no sharp corners), better surface finish.

Challenge: More complex interpolation, requires look-ahead for feedrate planning.

Path Tolerance and Contouring Accuracy

Path Tolerance

Definition: Maximum allowed deviation from programmed path.

Example: G1 move from (0,0) to (10,10) - Ideal: Straight line - Actual: Servo following errors cause deviation - Path tolerance: Maximum allowed deviation (e.g., 0.001")

Factors Affecting Path Accuracy: 1. **Following error:** Position lag during motion 2. **Corner blending:** Rounding of sharp corners (Section 19.9) 3. **Interpolation resolution:** Time step size 4. **Mechanical compliance:** Frame/screw deflection under load

Contouring Error

Definition: Perpendicular distance from actual position to desired path.

Example: Circular arc - Programmed: Perfect circle, radius = 5.000" - Actual: Slightly oval due to following errors - Contouring error: Radial deviation from circle

Measurement: - Circular interpolation test (G2/G3 around circle) - Measure radius at multiple points - Maximum deviation = contouring error

Typical Specifications: - Hobby CNC: 0.005-0.010" contouring error - Industrial CNC: 0.001-0.002" contouring error - Precision CNC: 0.0001-0.0005" contouring error

Improvement Methods: 1. Better servo tuning (reduce following error) 2. Feedforward control (FF1, FF2) 3. Slower feedrates (less dynamic error) 4. Stiffer mechanical construction

Blending vs. Exact Stop Mode

Exact Stop Mode

Behavior: Axis decelerates to complete stop at each programmed point.

G-Code: G61 (Exact Stop Mode)

G61

G1 X10 Y0 F100

G1 X10 Y10

G1 X0 Y10

Motion: 1. Move to (10, 0), **stop completely** 2. Move to (10, 10), **stop completely** 3. Move to (0, 10), **stop completely**

Advantages: - Guaranteed position accuracy at each point - Predictable (no corner rounding)

Disadvantages: - Slow (stop/start at every point) - Harsh (acceleration spikes at corners) - Poor surface finish (start/stop marks)

When to Use: - Probing operations - Precision positioning - Tool changes - When exact final position critical

Blending Mode (Constant Velocity)

Behavior: Axis maintains velocity through programmed points (rounds corners).

G-Code: G64 (Blending Mode)

G64

G1 X10 Y0 F100

G1 X10 Y10

G1 X0 Y10

Motion: 1. Move toward (10, 0) 2. **Before reaching (10,0)**, begin transitioning to next move 3. Round corner at (10, 0), never stop 4. Continue through (10, 10) with rounded corner 5. Slow down only at final point (0, 10)

Advantages: - Fast (no stops) - Smooth motion (no acceleration spikes) - Better surface finish (continuous motion)

Disadvantages: - Position error at corners (path deviation) - Requires look-ahead (know next move in advance)

When to Use: - Cutting operations (milling, routing) - 3D contouring - Any continuous path where exact corner position not critical

Blending with Tolerance (G64 P)

G-Code: G64 P[tolerance]

G64 P0.005

G1 X10 Y0 F100

G1 X10 Y10

Behavior: Blend corners, but limit path deviation to specified tolerance.

Example: - P0.005: Maximum 0.005" deviation from programmed corner - Tighter corners: Slow down to stay within tolerance - Gradual corners: Maintain high speed

Best of Both: Speed of blending + controlled accuracy.

Typical Values: - Roughing: P0.010-0.020 (fast, low accuracy needed) - Finishing: P0.001-0.005 (balance speed and finish) - Precision: P0.0001-0.001 (slow, high accuracy)

Summary

Trajectory planning bridges the gap between G-code commands and real-time motion:

Key Concepts: 1. **Path vs. Trajectory:** Geometry vs. time-based motion 2. **Kinematic Constraints:** Velocity, acceleration, jerk limits 3. **Interpolation:** Linear, circular, spline motion generation 4. **Blending:** Trade-off between speed and accuracy

Controller Responsibilities: - Generate smooth trajectories respecting all constraints - Coordinate multiple axes for accurate path following - Optimize feedrate for minimum cycle time - Provide real-time adjustability (feed override, feedhold)

Next Steps: - Learn motion profile design (Section 19.7) - Understand multi-axis coordination (Section 19.8) - Implement look-ahead and path blending (Section 19.9)

Next: 19.7 Motion Profiles

19.7 Motion Profiles

What is a Motion Profile?

Motion Profile: The velocity (and acceleration) vs. time curve for a single move.

Purpose: Define how system accelerates from rest, maintains velocity, and decelerates to target position while respecting kinematic constraints.

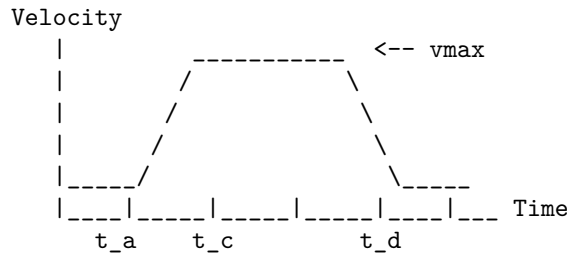
Profile Types: 1. **Trapezoidal:** Constant acceleration (simple, widely used) 2. **S-curve:** Jerk-limited (smooth, better for high-speed) 3. **Polynomial:** Higher-order (optimal for specific criteria) 4. **Bang-bang:** Minimum time (aggressive, rarely used in CNC)

Trapezoidal Velocity Profile

Profile Structure

Three Phases: 1. **Acceleration:** Constant acceleration from 0 to v_{max} 2. **Constant Velocity:** Cruise at v_{max} 3. **Deceleration:** Constant deceleration from v_{max} to 0

Velocity vs. Time:



Where: - t_a = acceleration time - t_c = constant velocity (cruise) time - t_d = deceleration time - v_{max} = maximum velocity (cruise velocity)

Mathematical Formulation

Given: - Distance: d (total move distance) - Max velocity: v_{max} - Max acceleration: a_{max}

Calculate Acceleration Time:

$$t_a = \frac{v_{max}}{a_{max}}$$

Distance During Acceleration:

$$d_a = \frac{1}{2} a_{max} t_a^2 = \frac{v_{max}^2}{2a_{max}}$$

Distance During Deceleration (assuming same deceleration rate):

$$d_d = d_a = \frac{v_{max}^2}{2a_{max}}$$

Distance During Cruise:

$$d_c = d - d_a - d_d$$

Cruise Time:

$$t_c = \frac{d_c}{v_{max}}$$

Total Time:

$$T = t_a + t_c + t_d = 2t_a + t_c$$

Example Calculation

Given: - Distance: $d = 10$ inches - Max velocity: $v_{max} = 200 \text{ IPM} = 3.33 \text{ in/s}$ - Max acceleration: $a_{max} = 100 \text{ in/s}^2$

Calculate: - $t_a = 3.33/100 = 0.0333 \text{ s}$ - $d_a = 3.33^2/(2 \times 100) = 0.0555 \text{ inches}$ - $d_d = 0.0555 \text{ inches}$ - $d_c = 10 - 0.0555 - 0.0555 = 9.889 \text{ inches}$ - $t_c = 9.889/3.33 = 2.969 \text{ s}$ - $T = 0.0333 + 2.969 + 0.0333 = 3.035 \text{ s}$

Trajectory:

Phase	Time (s)	Velocity (in/s)	Position (in)
Start	0.000	0.00	0.000
Accel	0.0333	3.33	0.056
Cruise	3.002	3.33	9.944
Decel	3.035	0.00	10.000

Short Move (Triangular Profile)

Problem: If move distance too short, never reaches v_{max} .

Condition: $d < \frac{v_{max}^2}{a_{max}}$ (total distance less than accel + decel distance)

Solution: **Triangular profile** (accelerate, then immediately decelerate)

Peak Velocity:

$$v_{peak} = \sqrt{a_{max} \cdot d}$$

Example: - Distance: $d = 0.5$ inches - Max acceleration: $a_{max} = 100 \text{ in/s}^2$ - Check: $\frac{v_{max}^2}{a_{max}} = 3.33^2/100 = 0.111 \text{ inches}$ - Since $0.5 > 0.111$, **trapezoidal profile OK** (will reach v_{max})

Short Example: - Distance: $d = 0.05$ inches - Since $0.05 < 0.111$, **triangular profile** - $v_{peak} = \sqrt{100 \times 0.05} = 2.24 \text{ in/s}$ (never reaches 3.33 in/s) - $t_a = 2.24/100 = 0.0224 \text{ s}$ - $T = 2 \times t_a = 0.0448 \text{ s}$

Advantages and Disadvantages

Advantages: - Simple to compute - Well-understood - Minimal computation (real-time friendly) - Predictable

Disadvantages: - Infinite jerk (instant acceleration change) - Excites mechanical resonances - Harsh motion (vibration, noise) - Poor surface finish in some applications

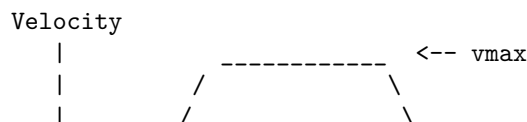
When to Use: - General-purpose machining - Robust mechanical systems (stiff, well-damped) - When cycle time critical (fastest profile type)

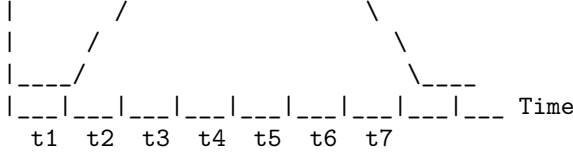
S-Curve (Jerk-Limited) Velocity Profile

Profile Structure

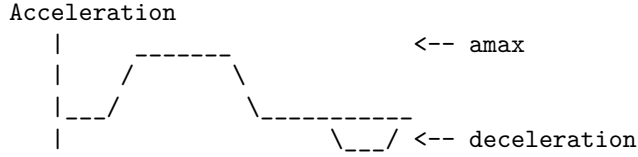
Seven Phases: 1. **Jerk-in** (acceleration increasing) 2. **Constant acceleration** 3. **Jerk-out** (acceleration decreasing to zero) 4. **Constant velocity** (cruise) 5. **Jerk-in** (deceleration starting) 6. **Constant deceleration** 7. **Jerk-out** (deceleration decreasing to zero)

Velocity vs. Time (S-curve shape):





Acceleration vs. Time:



Jerk vs. Time:



Mathematical Formulation

Given: - Distance: d - Max velocity: v_{max} - Max acceleration: a_{max} - Max jerk: j_{max}

Jerk Phase Duration:

$$t_j = \frac{a_{max}}{j_{max}}$$

Acceleration Phase Segments: - Jerk-in: $t_1 = t_j$ - Constant acceleration: t_2 (calculated) - Jerk-out: $t_3 = t_j$

Velocity at End of Jerk-In:

$$v_1 = \frac{1}{2} j_{max} t_j^2 = \frac{a_{max}^2}{2 j_{max}}$$

Total Acceleration Phase:

$$t_a = t_1 + t_2 + t_3 = 2t_j + t_2$$

Velocity After Acceleration Phase:

$$v_{max} = v_1 + a_{max} t_2 + v_1 = \frac{a_{max}^2}{j_{max}} + a_{max} t_2$$

Solve for t_2 :

$$t_2 = \frac{v_{max}}{a_{max}} - \frac{a_{max}}{j_{max}}$$

Distance During Acceleration:

$$d_a = \frac{1}{6} j_{max} t_1^3 + v_1 t_2 + \frac{1}{2} a_{max} t_2^2 + v_1 t_2 + \frac{1}{2} a_{max} t_2^2 + (\text{jerk-out contribution})$$

(Full equation complex; typically computed numerically)

Simplified (for symmetric accel/decel):

$$d_a \approx \frac{v_{max}^2}{2a_{max}} + \frac{a_{max}^2}{j_{max}}$$

Example Calculation

Given: - Distance: $d = 10$ inches - Max velocity: $v_{max} = 200$ IPM = 3.33 in/s - Max acceleration: $a_{max} = 100$ in/s² - Max jerk: $j_{max} = 1000$ in/s³

Calculate: - $t_j = 100/1000 = 0.1$ s - $v_1 = 100^2/(2 \times 1000) = 5.0$ in/s²*s = ... (check units: should be velocity) - Actually: $v_1 = \frac{1}{2} \times 1000 \times 0.1^2 = 5$ in/s - $t_2 = 3.33/100 - 100/1000 = 0.0333 - 0.1 = -0.0667$ s (negative! Problem)

Interpretation: Jerk limit too low; cannot reach v_{max} before jerk phase ends.

Adjust: Either increase j_{max} or accept lower peak velocity.

Revised: $j_{max} = 5000$ in/s³ - $t_j = 100/5000 = 0.02$ s - $v_1 = 0.5 \times 5000 \times 0.02^2 = 1.0$ in/s - $t_2 = 3.33/100 - 100/5000 = 0.0333 - 0.02 = 0.0133$ s (OK!) - $t_a = 2 \times 0.02 + 0.0133 = 0.0533$ s

Distance During Acceleration (approximate):

$$d_a \approx \frac{3.33^2}{2 \times 100} + \frac{100^2}{5000} = 0.0555 + 2.0 = 2.056$$

inches

Note: Exact calculation requires numerical integration; controllers compute this iteratively.

Advantages and Disadvantages

Advantages: - Smooth motion (limited jerk) - Reduced vibration and resonance excitation - Better surface finish - Less mechanical stress (bearings, frame) - Quieter operation

Disadvantages: - More complex computation (but manageable) - Slightly longer cycle time (jerk phases add time) - More parameters to tune (j_{max})

When to Use: - High-speed machining (vibration-sensitive) - Lightweight/flexible structures - Better surface finish desired - Noise reduction important

Typical Applications: - 3D printers (eliminates ringing) - High-speed routers - Pick-and-place machines - Laser cutters

Polynomial Trajectories

Concept

Polynomial Trajectory: Position as polynomial function of time.

General Form:

$$s(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + \dots + a_n t^n$$

Derivatives: - Velocity: $v(t) = \dot{s}(t) = a_1 + 2a_2 t + 3a_3 t^2 + \dots$ - Acceleration: $a(t) = \ddot{s}(t) = 2a_2 + 6a_3 t + \dots$ - Jerk: $j(t) = \dddot{s}(t) = 6a_3 + \dots$

Cubic Polynomial (3rd Order)

Application: Point-to-point move with specified start/end conditions.

Boundary Conditions: - $s(0) = 0$ (start position) - $s(T) = d$ (end position) - $v(0) = 0$ (start velocity) - $v(T) = 0$ (end velocity)

Polynomial:

$$s(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

Solve for Coefficients: From boundary conditions: - $a_0 = 0$ - $a_1 = 0$ - $a_2 = \frac{3d}{T^2}$ - $a_3 = -\frac{2d}{T^3}$

Result:

$$s(t) = \frac{3d}{T^2}t^2 - \frac{2d}{T^3}t^3 = d \left(3 \left(\frac{t}{T} \right)^2 - 2 \left(\frac{t}{T} \right)^3 \right)$$

Velocity:

$$v(t) = \frac{6d}{T^2}t - \frac{6d}{T^3}t^2$$

Peak Velocity (at $t = T/2$):

$$v_{peak} = \frac{6d}{T^2} \frac{T}{2} - \frac{6d}{T^3} \frac{T^2}{4} = \frac{3d}{T} - \frac{3d}{2T} = \frac{3d}{2T}$$

Acceleration:

$$a(t) = \frac{6d}{T^2} - \frac{12d}{T^3}t$$

Peak Acceleration (at $t = 0$ and $t = T$):

$$a_{peak} = \frac{6d}{T^2}$$

Example: - Distance: $d = 10$ inches - Move time: $T = 3$ seconds - $v_{peak} = (3 \times 10)/(2 \times 3) = 5$ in/s - $a_{peak} = (6 \times 10)/3^2 = 6.67$ in/s²

Quintic Polynomial (5th Order)

Boundary Conditions (more constraints): - $s(0) = 0$, $s(T) = d$ - $v(0) = 0$, $v(T) = 0$ - $a(0) = 0$, $a(T) = 0$ (smooth start and stop)

Polynomial:

$$s(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

Coefficients (derived from boundary conditions):

$$s(t) = d \left(10 \left(\frac{t}{T} \right)^3 - 15 \left(\frac{t}{T} \right)^4 + 6 \left(\frac{t}{T} \right)^5 \right)$$

Advantage: Zero acceleration at start and end (even smoother than cubic).

Peak Velocity (at $t = T/2$):

$$v_{peak} = \frac{15d}{8T}$$

Peak Acceleration:

$$a_{peak} = \frac{30d}{T^2} \text{ (at specific times, not start/end)}$$

When to Use: - Ultra-smooth motion required - Minimizing jerk - Synchronizing with other systems (robotics)

Minimum-Time Trajectories

Bang-Bang Control

Concept: Maximum acceleration, then maximum deceleration (no cruise phase).

Profile: - Accelerate at a_{max} for time t_a - Immediately decelerate at a_{max} for time $t_d = t_a$

Minimum Time:

$$T_{min} = 2\sqrt{\frac{d}{a_{max}}}$$

Example: - Distance: $d = 10$ inches - Max acceleration: $a_{max} = 100 \text{ in/s}^2$ - $T_{min} = 2\sqrt{10/100} = 2 \times 0.316 = 0.632$ seconds

Compare to Trapezoidal (with cruise at 200 IPM): - Trapezoidal time: 3.035 seconds (from earlier example) - **Bang-bang 4.8x faster** (but exceeds velocity limit!)

Reality Check: Must respect velocity limit. - Peak velocity (bang-bang): $v_{peak} = a_{max}t_a = 100 \times 0.316 = 31.6 \text{ in/s} = 1896 \text{ IPM}$ - Limit: 200 IPM = 3.33 in/s - **Cannot use bang-bang** (violates velocity constraint)

Application: Very short moves where v_{max} not reached (triangular profile approx bang-bang).

Optimal Time with Constraints

Problem: Find fastest trajectory respecting v_{max} , a_{max} , j_{max} .

Solution: Typically trapezoidal or S-curve (depending on jerk limit).

Algorithm: 1. Assume trapezoidal, check if v_{max} reached 2. If not, triangular profile (minimum time for given a_{max}) 3. If jerk limit active, S-curve profile 4. Check all constraints; reduce velocity/acceleration as needed

Modern CNC Controllers: Automatically compute near-optimal trajectories using look-ahead and constraint checking.

Motion Profile Selection

Application-Based Selection

High-Speed Machining: - **Profile:** S-curve (jerk-limited) - **Reason:** Smooth motion, reduced vibration, better finish - **Typical:** $j_{max} = 50,000\text{-}200,000 \text{ in/s}^3$

Heavy Machining: - **Profile:** Trapezoidal (simple) - **Reason:** Stiff machine, cutting forces dominate, simplicity preferred - **Typical:** Standard acceleration limits

Precision Positioning: - **Profile:** Polynomial (quintic) or S-curve - **Reason:** Smooth, no jerks, repeatable - **Typical:** Slow, controlled moves

Rapid Positioning: - **Profile:** Trapezoidal (maximum speed) - **Reason:** Speed priority, no cutting - **Typical:** High a_{max} , high v_{max}

Trade-Offs Summary

Profile	Cycle Time	Smoothness	Complexity	Surface Finish
Trapezoidal	Fastest	Harsh	Simple	Good
S-curve	Moderate	Smooth	Moderate	Excellent
Cubic Poly	Slow	Very Smooth	Moderate	Excellent
Quintic Poly	Slowest	Smoothest	Complex	Excellent

Velocity Profile Optimization

Adaptive Feedrate

Concept: Adjust feedrate based on cutting conditions.

Inputs: - Cutting force measurement (dynamometer or motor current) - Surface finish requirements - Tool wear state

Algorithm: 1. Monitor cutting force 2. If force > threshold: Reduce feedrate 3. If force < threshold: Increase feedrate 4. Stay within v_{max} and a_{max}

Example: - Programmed feedrate: 100 IPM - Heavy cut detected: Reduce to 70 IPM (temporary) - Light cut: Restore to 100 IPM

Application: Optimizing cycle time while preventing tool breakage.

Constant Surface Speed (CSS)

Lathe Application: Maintain constant cutting speed at tool edge as diameter changes.

Formula:

$$v = \frac{\pi DN}{12}$$

where: - v = surface speed (SFM) - D = workpiece diameter (inches) - N = spindle RPM

Maintain Constant v : Adjust N as D changes.

Example: - Target surface speed: 300 SFM - At $D = 4$ ": $N = (300 \times 12) / (\pi \times 4) = 286$ RPM - At $D = 2$ ": $N = (300 \times 12) / (\pi \times 2) = 573$ RPM

G-Code: G96 (Constant Surface Speed mode)

Trajectory Planning: Controller adjusts spindle speed profile synchronized with motion.

Summary

Motion profiles define velocity vs. time for moves:

Key Profile Types: 1. **Trapezoidal:** Simple, fast, harsh (infinite jerk) 2. **S-curve:** Smooth, jerk-limited, best for high-speed 3. **Polynomial:** Very smooth, flexible boundary conditions

Selection Criteria: - **Cycle time:** Trapezoidal fastest - **Surface finish:** S-curve or polynomial best - **Mechanical stress:** S-curve reduces wear and vibration

Modern Controllers: Automatically generate near-optimal profiles based on constraints.

Next Steps: - Multi-axis coordination (Section 19.8) - Look-ahead and path blending (Section 19.9) - Implementation in LinuxCNC/Mach4 (Sections 19.10-19.11)

Next: 19.8 Multi-Axis Coordination

19.8 Multi-Axis Coordination

The Coordination Challenge

Single-Axis Motion: Straightforward—move one axis from point A to point B.

Multi-Axis Motion: Complex—multiple axes must move simultaneously in coordinated fashion to: - Follow precise geometric path (lines, arcs, surfaces) - Maintain programmed feedrate along path - Arrive at target position simultaneously - Respect individual axis constraints

Example Problem: Move from (0,0) to (10,5) in straight line - If X and Y move independently at max speed: - X reaches 10 at t_1 - Y reaches 5 at t_2 (where $t_2 \neq t_1$) - Actual path is NOT straight line!

Solution: Coordinate X and Y motion so they reach target simultaneously while maintaining geometric path.

Synchronized Motion Basics

Time Synchronization

Principle: All axes complete move in same total time.

Algorithm: 1. Calculate path length: $L = \sqrt{\Delta X^2 + \Delta Y^2 + \Delta Z^2}$ 2. Determine move time: $T = L/F$ (where F = programmed feedrate) 3. Calculate individual axis velocities: - $v_X = \Delta X/T$ - $v_Y = \Delta Y/T$ - $v_Z = \Delta Z/T$ 4. Generate synchronized motion profiles

Example: - Start: (0, 0, 0) - End: (10, 5, 2) - Feedrate: $F = 120 \text{ IPM} = 2 \text{ in/s}$ - Path length: $L = \sqrt{10^2 + 5^2 + 2^2} = \sqrt{129} = 11.36 \text{ inches}$ - Move time: $T = 11.36/2 = 5.68 \text{ seconds}$ - Axis velocities: - $v_X = 10/5.68 = 1.76 \text{ in/s}$ - $v_Y = 5/5.68 = 0.88 \text{ in/s}$ - $v_Z = 2/5.68 = 0.35 \text{ in/s}$

Verification: $\sqrt{v_X^2 + v_Y^2 + v_Z^2} = \sqrt{1.76^2 + 0.88^2 + 0.35^2} = 2.0 \text{ in/s}$ [check]

Velocity Vector Coordination

Tool Center Point (TCP) Velocity: Velocity along programmed path.

$$\vec{v}_{TCP} = \frac{d\vec{r}}{dt}$$

where $\vec{r}(t) = [X(t), Y(t), Z(t)]$ = position vector

Components:

$$\vec{v}_{TCP} = [v_X, v_Y, v_Z]$$

Magnitude (feedrate):

$$|\vec{v}_{TCP}| = \sqrt{v_X^2 + v_Y^2 + v_Z^2} = F$$

Controller Responsibility: Continuously adjust individual axis velocities to maintain: 1. $|\vec{v}_{TCP}| = F$ (constant feedrate along path) 2. Direction along programmed path

Linear Interpolation (Multi-Axis)

Three-Axis Linear Move

G-Code:

G1 X10 Y5 Z-2 F150

Trajectory Generation (each servo cycle):

Step 1: Calculate unit direction vector

$$\hat{u} = \frac{[\Delta X, \Delta Y, \Delta Z]}{L} = \frac{[10, 5, -2]}{11.36} = [0.880, 0.440, -0.176]$$

Step 2: Calculate distance traveled (from velocity profile) - At time t , total distance along path: $s(t)$ (from motion profile)

Step 3: Calculate individual axis positions

$$X(t) = X_0 + \hat{u}_X \cdot s(t)$$

$$Y(t) = Y_0 + \hat{u}_Y \cdot s(t)$$

$$Z(t) = Z_0 + \hat{u}_Z \cdot s(t)$$

Example (at $t = 2$ seconds, $s(t) = 4.0$ inches): - $X(2) = 0 + 0.880 \times 4.0 = 3.52$ inches - $Y(2) = 0 + 0.440 \times 4.0 = 1.76$ inches - $Z(2) = 0 + (-0.176) \times 4.0 = -0.70$ inches

Position commands sent to each axis PID loop every servo cycle (e.g., 1 ms).

Feedrate Along Path

Programmed Feedrate: $F = 150$ IPM (in this example)

Actual TCP Velocity (instantaneous):

$$v_{TCP}(t) = \frac{ds(t)}{dt}$$

During Acceleration: - v_{TCP} increases from 0 to F (ramping up) - Individual axis velocities increase proportionally

During Cruise: - $v_{TCP} = F$ (constant) - Individual axis velocities constant

During Deceleration: - v_{TCP} decreases from F to 0 - Individual axis velocities decrease proportionally

Key Point: Ratio of axis velocities remains constant (maintains straight line path).

Circular Interpolation (Multi-Axis)

Two-Axis Circular Motion (XY Plane)

G-Code:

G2 X10 Y10 I5 J0 F100

Parameters: - Current position: (X_0, Y_0) - Target: $(X_1, Y_1) = (10, 10)$ - Center offset: $(I, J) = (5, 0)$ - Arc center: $(X_c, Y_c) = (X_0 + I, Y_0 + J)$

Trajectory Generation:

Step 1: Calculate arc parameters - Radius: $R = \sqrt{I^2 + J^2}$ - Start angle: $\theta_0 = \text{atan2}(Y_0 - Y_c, X_0 - X_c)$ - End angle: $\theta_1 = \text{atan2}(Y_1 - Y_c, X_1 - X_c)$ - Arc length: $L = R|\theta_1 - \theta_0|$

Step 2: Calculate instantaneous angle

$$\theta(t) = \theta_0 + \frac{s(t)}{R}$$

where $s(t)$ = arc distance traveled

Step 3: Calculate X, Y positions

$$X(t) = X_c + R \cos(\theta(t))$$

$$Y(t) = Y_c + R \sin(\theta(t))$$

Step 4: Calculate velocities (derivatives)

$$v_X(t) = -R \sin(\theta(t)) \cdot \frac{d\theta}{dt}$$

$$v_Y(t) = R \cos(\theta(t)) \cdot \frac{d\theta}{dt}$$

where $\frac{d\theta}{dt} = \frac{v_{TCP}(t)}{R}$ (angular velocity)

Centripetal Acceleration Constraint

Problem: Circular motion creates centripetal acceleration.

$$a_c = \frac{v^2}{R}$$

Example: - Radius: $R = 2$ inches - Feedrate: $F = 200$ IPM = 3.33 in/s - Centripetal acceleration: $a_c = 3.33^2/2 = 5.55$ in/s²

If $a_c > a_{max}$: Controller must reduce feedrate.

Maximum Allowable Feedrate:

$$v_{max} = \sqrt{a_{max} \cdot R}$$

Example (small radius): - Radius: $R = 0.1$ inches - Max acceleration: $a_{max} = 200$ in/s² - $v_{max} = \sqrt{200 \times 0.1} = 4.47$ in/s = 268 IPM

If programmed $F = 400$ IPM, controller automatically reduces to 268 IPM for this arc.

Feedrate Override for Arcs: Modern controllers automatically compute this; programmer doesn't need to manually adjust F for every arc radius.

Helical Interpolation (3-Axis Circular)

G-Code:

G2 X10 Y10 Z-5 I5 J0 F100

Motion: - XY plane: Circular arc - Z axis: Simultaneous linear motion

Trajectory:

$$X(t) = X_c + R \cos(\theta(t))$$

$$Y(t) = Y_c + R \sin(\theta(t))$$

$$Z(t) = Z_0 + \frac{\Delta Z}{L_{arc}} \cdot s(t)$$

where: - L_{arc} = arc length in XY plane - $s(t)$ = distance traveled along arc

Feedrate: Along 3D helical path (not just XY arc)

$$L_{total} = \sqrt{L_{arc}^2 + \Delta Z^2}$$

Application: Thread milling, helical entry into holes.

Tool Center Point (TCP) Control

TCP Definition

Tool Center Point: The effective point of the tool (e.g., tip of end mill, center of ball on CMM probe).

CNC Context: Control TCP position, not machine coordinate position.

Why TCP Control Matters: - Tool length varies (different tools, tool wear) - Work offset (part location on table) - Rotary axes (A, B, C) change TCP location relative to machine coordinates

Tool Length Compensation (TLC)

Problem: Different tools have different lengths.

Solution: Define tool length offset; controller adjusts Z position.

G-Code:

G43 H1 Z0.5 ; Activate tool length comp, use offset H1

Controller Calculation:

$$Z_{machine} = Z_{programmed} + \text{ToolOffset}[H1]$$

Example: - Programmed Z = 0.5" (above part surface) - Tool offset H1 = -3.5" (tool length) - Machine Z = $0.5 + (-3.5) = -3.0$ " (absolute machine position)

TCP Position: 0.5" above part surface (regardless of tool length)

Multi-Axis TCP (5-Axis Machining)

5-Axis Configuration: X, Y, Z (linear) + A, B (rotary)

Challenge: Rotary axis motion changes TCP position.

Example: Tilt B-axis by 10 deg - Tool tip position changes in X and Z - Must coordinate X, Z motion to keep TCP stationary during B rotation

Forward Kinematics: Calculate TCP position from joint (axis) positions

$$\vec{r}_{TCP} = f(X, Y, Z, A, B)$$

Inverse Kinematics: Calculate required joint positions for desired TCP position

$$[X, Y, Z, A, B] = f^{-1}(\vec{r}_{TCP}, \text{orientation})$$

Controller Responsibility: Solve inverse kinematics in real-time; coordinate all 5 axes.

Complexity: Non-trivial mathematics; requires 5-axis controller (not all CNC controllers support).

Coordinated Motion Constraints

Individual Axis Limits

Each Axis Has Limits: - Maximum velocity: $v_{max,i}$ - Maximum acceleration: $a_{max,i}$ - Maximum jerk: $j_{max,i}$

Coordinated Motion: Must respect ALL axis limits simultaneously.

Velocity Constraint Checking

Given: Desired feedrate F along path

Required Axis Velocities:

$$v_i = F \cdot \frac{\Delta r_i}{L}$$

where: - Δr_i = motion distance for axis i - L = path length

Constraint Check:

$$|v_i| \leq v_{max,i} \text{ for all } i$$

If Violated: Reduce F to maximum allowable value.

Maximum Allowable F :

$$F_{max} = \min_i \left(v_{max,i} \cdot \frac{L}{|\Delta r_i|} \right)$$

Example: - Move: DeltaX = 10", DeltaY = 10", DeltaZ = 5" - Path length: $L = \sqrt{10^2 + 10^2 + 5^2} = 15$ inches - Programmed $F = 600$ IPM = 10 in/s - Axis limits: - $v_{max,X} = 600$ IPM = 10 in/s - $v_{max,Y} = 500$ IPM = 8.33 in/s - $v_{max,Z} = 300$ IPM = 5 in/s

Required Velocities: - $v_X = 10 \times (10/15) = 6.67$ in/s (OK, < 10) - $v_Y = 10 \times (10/15) = 6.67$ in/s (OK, < 8.33) - $v_Z = 10 \times (5/15) = 3.33$ in/s (OK, < 5)

All constraints satisfied: $F = 600$ IPM achievable.

Example 2 (constraint violated): - Move: DeltaX = 10", DeltaY = 10", DeltaZ = 10" - Path length: $L = 17.32$ inches - Programmed $F = 800$ IPM = 13.33 in/s - Required $v_Z = 13.33 \times (10/17.32) = 7.70$ in/s (exceeds 5 in/s limit!)

Maximum Allowable F :

$$F_{max} = 5.0 \times \frac{17.32}{10} = 8.66 \text{ in/s} = 520 \text{ IPM}$$

Controller automatically reduces F to 520 IPM.

Acceleration Constraint Checking

Similar Process: Check that required axis accelerations don't exceed limits.

During Velocity Change: Each axis accelerates proportionally.

$$a_i = \frac{a_{TCP} \cdot \Delta r_i}{L}$$

where a_{TCP} = acceleration along path

Constraint:

$$|a_i| \leq a_{max,i} \text{ for all } i$$

If Violated: Reduce a_{TCP} (slower ramp-up/down).

Contouring Accuracy

Path Following Error

Ideal: Tool follows programmed path exactly

Reality: Following errors cause deviation from path

Contouring Error: Perpendicular distance from actual TCP to ideal path

Sources of Contouring Error

1. **Individual Axis Following Errors:**
 - Each axis lags command by small amount
 - Combined effect: TCP deviates from path
2. **Servo Tuning Mismatch:**
 - X-axis responds faster than Y-axis
 - Path distortion (even if following errors small)
3. **Acceleration/Deceleration:**
 - Transient errors during speed changes
 - Worse at corners (large acceleration)
4. **Mechanical Compliance:**
 - Cutting forces deflect tool
 - Direction-dependent (different X vs. Y stiffness)

Reducing Contouring Error

Method 1: Better Servo Tuning - Minimize individual following errors (PID + feedforward) - Match response of all axes (similar bandwidth)

Method 2: Cross-Coupling Control - Measure contouring error directly - Apply correction to maintain path accuracy - More complex, not common in standard CNC controllers

Method 3: Slower Feedrates - Reduces dynamic errors (acceleration-dependent) - Trade-off: Longer cycle time

Method 4: Mechanical Improvements - Stiffer frame (reduce compliance) - Better bearings (reduce friction variation) - Balanced axes (similar inertia, friction)

Circular Test (Contouring Accuracy Check)

Procedure: 1. Mount dial indicator or probe 2. Program circular interpolation (e.g., 4" diameter circle) 3. Measure actual radius at multiple points 4. Deviation from nominal = contouring error

Example Results: - Nominal radius: 2.000" - Measured: 1.998" to 2.003" (variation = 0.005") - **Contouring error: +/-0.0025" (radial)**

Typical Specifications: - Hobby CNC: +/-0.005-0.010" - Industrial CNC: +/-0.001-0.002" - Precision CNC: +/-0.0001-0.0005"

Gantry Synchronization (Dual-Motor Axis)

The Gantry Problem

Dual-Motor Gantry (common on plasma tables, large routers): - Two motors drive same axis (e.g., left and right side of Y-axis gantry) - Motors must stay perfectly synchronized - Mismatch causes gantry to rack (skew)

Consequences of Racking: - Binding (mechanical stress) - Loss of accuracy - Premature wear - Possible motor stall

Independent Motor Control

Approach 1: Treat as separate axes (e.g., Y1 and Y2)

G-Code Coordination:

G1 Y10 ; Both Y1 and Y2 commanded to move 10"

Problem: If one motor lags (even slightly), gantry racks.

Example: - Y1 position: 10.002" - Y2 position: 9.998" - **Racking error: 0.004"** (across gantry width)

Slaving (Simple Synchronization)

Approach 2: One motor is "master", other is "slave"

Configuration: - Y1 = master (receives position commands from controller) - Y2 = slave (copies Y1 commands)

Problem: Still no correction if slave lags (open-loop slaving)

Cross-Coupling Control (Active Synchronization)

Approach 3: Feedback from both motors, cross-coupling controller

Algorithm: 1. Measure positions: y_1, y_2 2. Calculate average: $y_{avg} = (y_1 + y_2)/2$ 3. Calculate sync error: $e_{sync} = y_1 - y_2$ 4. Commands: - $u_1 = \text{PID}(r - y_1) - K_{sync} \cdot e_{sync}$ - $u_2 = \text{PID}(r - y_2) + K_{sync} \cdot e_{sync}$

Effect: - If Motor 1 leads: Reduce u_1 , increase u_2 -> synchronize - If Motor 2 leads: Increase u_1 , reduce u_2 -> synchronize

Tuning K_{sync} : - Start: $K_{sync} = 0.1 \times K_P$ - Increase until $|e_{sync}| < 0.001$ " - Too high: Instability (motors fight each other)

LinuxCNC: gantrykins kinematics module handles this automatically.

Summary

Multi-axis coordination ensures all axes work together to follow programmed path:

Key Concepts: 1. **Time Synchronization:** All axes complete move simultaneously 2. **Velocity Coordination:** Individual axis velocities maintain path direction 3. **Constraint Checking:** Respect individual axis limits (velocity, acceleration) 4. **Contouring Accuracy:** Minimize perpendicular deviation from path

Interpolation Types: - **Linear:** Straight lines (3+ axes coordinated) - **Circular:** Arcs (2-3 axes, centripetal acceleration constrained) - **Helical:** 3D spiral (circular + linear combined)

Advanced Topics: - **TCP Control:** Account for tool length, rotary axes - **Gantry Sync:** Dual-motor coordination (cross-coupling)

Next Steps: - Look-ahead and path blending (Section 19.9) - Implementation in LinuxCNC (Section 19.10) - Implementation in Mach4 (Section 19.11)

Next: 19.9 Look-Ahead and Path Blending

19.9 Look-Ahead and Path Blending

The Look-Ahead Problem

Sequential G-Code Processing (naive approach): 1. Execute Line 1 completely 2. Stop at end of Line 1 3. Read Line 2 4. Execute Line 2 5. Repeat...

Problem: Stop-and-go motion - Slow (acceleration/deceleration at every line) - Harsh (jerky motion) - Poor surface finish (start/stop marks)

Solution: Look-Ahead - read multiple lines ahead, plan smooth continuous motion.

Why Look-Ahead is Necessary

Example G-Code (square profile):

```
N10 G1 X10 Y0 F100
N20 G1 X10 Y10
N30 G1 X0 Y10
N40 G1 X0 Y0
```

Without Look-Ahead: - Accelerate to F100, move to (10,0), **decelerate to stop** - **Stop completely** at corner - Accelerate to F100, move to (10,10), **decelerate to stop** - Repeat for each corner - **Total time:** ~4x longer due to stops

With Look-Ahead: - Read all 4 lines before starting - Plan continuous motion through corners - Never stop (except at final point) - **Total time:** Much faster, smooth motion

Look-Ahead Buffer

Buffer Structure

Read-Ahead Buffer: Queue of upcoming G-code blocks.

Typical Size: 50-200 blocks (depends on controller memory and computational power)

Example: - Controller at Line N50 - Buffer contains: N51, N52, ..., N150 - As N50 completes, N51 moves to execution, N151 added to buffer

Buffer Benefits

- 1. Corner Planning:** - Know next move direction before reaching corner - Calculate optimal corner velocity (blend vs. stop)
- 2. Velocity Optimization:** - Look ahead for tight corners, slow arcs - Preemptively reduce velocity - Avoid sudden decelerations
- 3. Smooth Acceleration Profiles:** - Plan acceleration/deceleration across multiple moves - Smoother than per-move planning
- 4. Constraint Checking:** - Check upcoming moves for axis limits - Adjust velocity proactively

Computational Challenge

Real-Time Constraint: Must plan trajectory faster than execution.

Example: - Feedrate: 100 IPM = 1.67 in/s - Line length: 0.1 inches - Execution time: $0.1 / 1.67 = 0.06$ seconds - **Planner must process line in <0.06 seconds**

Short Lines (common in CAM output): - 1000+ lines per second typical - Requires efficient algorithms (linear time, not exponential)

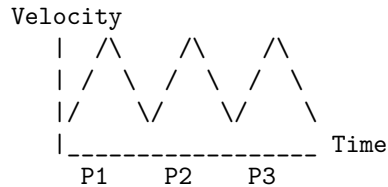
Modern Controllers: Dedicated trajectory planning processor/thread.

Corner Blending Strategies

Exact Stop (No Blending)

G61 Mode: Decelerate to complete stop at each programmed point.

Motion:



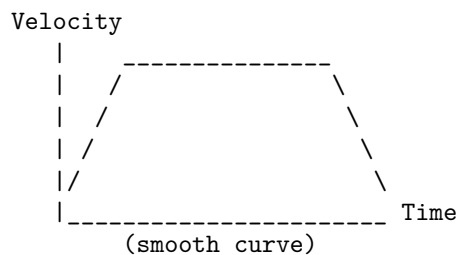
Advantages: - Exact position at every point - Predictable - Simple

Disadvantages: - Slow (stops at every point) - Jerky motion - Poor surface finish (start/stop marks)

Continuous Blending (G64)

G64 Mode: Maintain velocity through corners by rounding.

Motion:



Path Deviation: Actual path rounds corners (doesn't pass through exact programmed points).

Advantage: Fast, smooth motion.

Disadvantage: Position error at corners.

Tolerance-Based Blending (G64 P)

G64 P[tolerance]: Blend corners with maximum allowed path deviation.

Example:

```
G64 P0.005 ; Max 0.005" path deviation
G1 X10 Y0 F100
G1 X10 Y10
```

Algorithm: 1. Calculate corner angle 2. Determine blend radius for given tolerance 3. Slow down if necessary to stay within tolerance

Balance: Speed (large blend radius) vs. Accuracy (small blend radius).

Corner Velocity Calculation

Geometric Analysis

Two-Line Corner: - Line 1: Direction $\vec{u}_1$ - Line 2: Direction $\vec{u}_2$ - Corner angle: $\theta = \cos^{-1}(\vec{u}_1 \cdot \vec{u}_2)$

Corner Velocity (for given blend tolerance):

Sharp Corner (θ near 90 deg): - Requires low velocity (high direction change) - Large centripetal acceleration

Gradual Corner (θ near 180 deg): - Can maintain high velocity (slight direction change) - Small centripetal acceleration

Blend Radius and Tolerance

Blend Arc: Circular arc connecting two lines tangentially.

Chord Tolerance (path deviation):

$$P = r(1 - \cos(\theta/2))$$

where: - P = chord tolerance (max path deviation) - r = blend radius - θ = corner angle

Solve for Blend Radius:

$$r = \frac{P}{1 - \cos(\theta/2)}$$

Example: - Corner angle: $\theta = 90deg$ - Tolerance: $P = 0.010$ inches - Blend radius: $r = 0.010 / (1 - \cos(45deg)) = 0.010 / 0.293 = 0.034$ inches

Maximum Corner Velocity

Centripetal Acceleration Constraint:

$$a_c = \frac{v^2}{r} \leq a_{max}$$

Maximum Velocity:

$$v_{max} = \sqrt{a_{max} \cdot r}$$

Example: - Blend radius: $r = 0.034$ inches - Max acceleration: $a_{max} = 200 \text{ in/s}^2$ - $v_{max} = \sqrt{200 \times 0.034} = 2.61 \text{ in/s} = 156 \text{ IPM}$

Controller Action: If programmed $F = 200 \text{ IPM}$, reduce to 156 IPM for this corner.

Axis Acceleration Constraints

Additional Check: Individual axis accelerations during corner.

Example (90 deg XY corner): - Approaching along +X - Exiting along +Y - At corner apex: v_X decreasing, v_Y increasing - Peak acceleration: Both axes at a_{max} simultaneously?

Vector Sum:

$$\vec{a}_{total} = [a_X, a_Y]$$
$$|\vec{a}_{total}| = \sqrt{a_X^2 + a_Y^2}$$

For 90 deg corner: $|\vec{a}_{total}| = \sqrt{2} \cdot a_{max}$ (if both axes at limit)

Solution: Reduce corner velocity to keep $|\vec{a}_{total}| \leq a_{max}$.

Velocity Planning Along Path

Look-Ahead Velocity Profiling

Forward Pass (look ahead from start): 1. Start at programmed feedrate F 2. For each upcoming corner:
- Calculate max corner velocity v_{corner} - If $v_{corner} < v_{current}$: Begin decelerating - Calculate deceleration distance: $d = \frac{v_{current}^2 - v_{corner}^2}{2a_{max}}$ 3. Propagate constraints forward

Backward Pass (look back from end): 1. Start from final point ($v = 0$ or final feedrate) 2. For each previous segment:
- Calculate max velocity (considering next segment constraint) - Acceleration distance: $d = \frac{v_{next}^2}{2a_{max}}$ 3. Propagate constraints backward

Optimal Profile: Minimum of forward and backward passes.

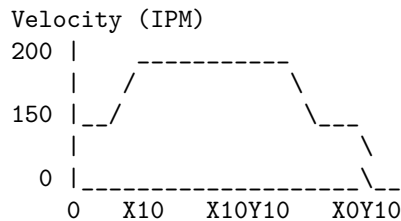
Example Velocity Profile

Scenario: Three lines forming two 90 deg corners

```
G64 P0.005
G1 X10 Y0 F200
G1 X10 Y10
G1 X0 Y10
```

Calculate Corner Velocities: - Corner 1 (10,0): $v_{c1} = 150$ IPM (from blend radius calculation) - Corner 2 (10,10): $v_{c2} = 150$ IPM

Velocity Profile:



Motion: 1. Accelerate to 200 IPM 2. Decelerate approaching Corner 1, reaching 150 IPM 3. Blend through Corner 1 at 150 IPM 4. Accelerate back to 200 IPM 5. Decelerate approaching Corner 2 6. Blend through Corner 2 at 150 IPM 7. Decelerate to stop at final point

Smooth, continuous motion with no complete stops.

Path Tolerance and Accuracy

Tolerance Specification

Typical Values: - **Roughing:** $P = 0.010$ - 0.050 " (fast, low accuracy) - **Finishing:** $P = 0.001$ - 0.005 " (balanced) - **Precision:** $P = 0.0001$ - 0.001 " (slow, high accuracy)

Trade-Off: Tighter tolerance = slower corners = longer cycle time.

Example: - With $P = 0.010$ " : Corner velocity = 200 IPM - With $P = 0.001$ " : Corner velocity = 80 IPM - **2.5x slower** for 10x tighter tolerance

Adaptive Tolerance

Concept: Vary tolerance based on operation.

Example Strategy: - Roughing passes: Loose tolerance ($P = 0.020$ ") - Semi-finishing: Medium ($P = 0.005$ ") - Finishing pass: Tight ($P = 0.001$ ")

G-Code:

```
; Roughing
G64 P0.020
(roughing toolpath)
```

```
; Finishing
G64 P0.001
(finishing toolpath)
```

Benefit: Optimize cycle time while achieving required final accuracy.

Acceleration Limiting

Jerk-Limited Acceleration

Problem: Instantaneous acceleration changes (trapezoidal profile) cause vibration.

Solution: Limit jerk (rate of acceleration change).

S-Curve Profile (covered in Section 19.7): - Smooth acceleration transitions - Requires look-ahead to plan jerk-limited ramps

Look-Ahead for Jerk: - Calculate jerk-limited acceleration distance - Ensure sufficient distance available before corner - If not, reduce velocity earlier

Coordinated Acceleration

Multi-Axis Constraint: All axes must respect acceleration limits.

Example (diagonal move with corner): - X and Y accelerating simultaneously - Combined acceleration vector: $\vec{a} = [a_X, a_Y]$ - Constraint: $|\vec{a}| \leq a_{max}$ (for each axis)

Look-Ahead Planner: Calculate required axis accelerations for path, reduce feedrate if any axis limit exceeded.

CAM Integration

G-Code Post-Processing

CAM Software: Generates toolpath geometry.

Post-Processor: Converts geometry to G-code.

Post-Processor Responsibilities: 1. Set appropriate G64 mode and tolerance 2. Insert feedrate changes for tight corners 3. Break complex curves into linear segments 4. Optimize move order for efficiency

Example (CAM Output):

```
G64 P0.002 ; Tight tolerance for finishing
G1 X0 Y0 Z0.1 F100
G1 X0.1 Y0
G1 X0.2 Y0.005
G1 X0.3 Y0.015
; ... (many short line segments approximating curve)
```

CNC Controller: Blends these short segments into smooth motion.

Line Segment Length

Tolerance vs. Segment Length: - Finer segments (0.001" long): Smoother curves, more processing - Coarser segments (0.010" long): Faster processing, chord error

Typical CAM Output: - Roughing: 0.010-0.050" segments - Finishing: 0.001-0.010" segments - High-precision: 0.0001-0.001" segments

Controller Challenge: Process thousands of short segments per second.

Modern Controllers (e.g., LinuxCNC): - Can handle 10,000+ segments/second with look-ahead - Real-time trajectory planning (1 kHz servo loop)

Feedhold and Feed Override

Feedhold (Stop in Place)

Function: Pause motion immediately while maintaining control.

Implementation with Look-Ahead: 1. Feedhold button pressed 2. Controller initiates deceleration (at a_{max}) 3. Motion stops smoothly 4. Position maintained (servos still active) 5. Resume: Accelerate back to programmed feedrate

Challenge: Look-ahead buffer contains future moves.

Solution: Controller tracks current position in buffer, resumes from correct point.

Feed Rate Override

Function: Adjust feedrate in real-time (e.g., 50%-150% of programmed F).

Example: - Programmed: F100 - Override: 120% - Actual: F120

Implementation: - Controller scales velocity profile by override factor - Look-ahead planner recomputes constraints in real-time - Smooth transition (ramp from old feedrate to new)

Challenge: Recompute corner velocities, acceleration profiles on-the-fly.

Modern Controllers: Handle override changes seamlessly (background recomputation).

Advanced Blending Techniques

Biarc Blending

Biarc: Two circular arcs (tangent to each other and to incoming/outgoing lines).

Advantage: Smoother than single circular arc (better approximation of optimal path).

Application: High-precision contouring (better than simple circular blend).

Complexity: More computation; not common in standard CNC controllers.

Spline Blending

Cubic Spline: Smooth curve (continuous to 2nd derivative).

Blending: Fit spline through multiple consecutive points.

Example:

```
G1 X1 Y0
G1 X2 Y1
G1 X3 Y1.5
G1 X4 Y1.8
```

Spline Blend: Single smooth curve through all 4 points (instead of 3 separate blends).

Advantage: Optimal smoothness, fewer acceleration changes.

Challenge: Requires advanced look-ahead (multiple moves at once).

Support: LinuxCNC (G5.1 cubic spline), limited in other controllers.

NURBS Interpolation

NURBS (Non-Uniform Rational B-Splines): Industry-standard CAD curve representation.

Direct NURBS G-Code (G5.2/G5.3): - CAM outputs NURBS parameters directly - Controller interpolates NURBS in real-time - No line segment approximation needed

Advantage: Perfect curve fidelity, minimal G-code size.

Disadvantage: Complex interpolation, limited controller support.

Status: Emerging technology (not widely supported yet).

Performance Benchmarking

Cycle Time Comparison

Test Part: Square with rounded corners (4 corners, 10" sides)

Exact Stop Mode (G61): - Stop at each corner: 4 stops - Acceleration time: 0.5 s per stop - Total stop time: $4 \times 0.5 \times 2 = 4$ seconds (accel + decel) - Cutting time: $40 / (100 \text{ IPM}) \times 60 = 24$ seconds - **Total: 28 seconds**

Blending Mode (G64): - No complete stops - Corner velocity: 80 IPM (from blend calculation) - Decel/accel time: ~ 0.2 s per corner - Total transition time: $4 \times 0.2 = 0.8$ seconds - Cutting time: ~ 24 seconds (varies with velocity profile) - **Total: ~ 25 seconds**

Savings: 3 seconds (11% reduction) for this simple part.

Complex Part (hundreds of corners): - Savings can be 30-50% with blending.

Surface Finish Comparison

Exact Stop: - Visible start/stop marks at corners - Scalloping from deceleration/acceleration - Rougher finish ($R_a = 100\text{-}200 \mu\text{in}$ typical)

Blending: - Smooth continuous motion - No start/stop marks - Better finish ($R_a = 50\text{-}100 \mu\text{in}$ typical) - May have slight corner rounding (within tolerance)

Summary

Look-ahead and path blending are essential for high-performance CNC:

Key Concepts: 1. **Look-Ahead Buffer:** Read multiple blocks ahead, plan smooth continuous motion 2. **Corner Blending:** Round corners to maintain velocity (G64 mode) 3. **Tolerance-Based Blending:** Balance speed and accuracy (G64 P) 4. **Velocity Planning:** Compute optimal feedrate considering all upcoming constraints

Benefits: - **30-50% faster** cycle times (vs. exact stop) - **Better surface finish** (no start/stop marks) - **Smoother motion** (less vibration, mechanical stress)

Trade-Offs: - Path deviation at corners (controlled by tolerance) - More complex controller (computational requirements) - Tuning needed (tolerance selection)

Modern CNC Controllers: Implement sophisticated look-ahead and blending automatically (LinuxCNC, Mach4, industrial controllers).

Next Steps: - Implement in LinuxCNC (Section 19.10) - Implement in Mach4 (Section 19.11) - Troubleshooting and optimization (Section 19.12)

Next: 19.10 Implementation in LinuxCNC